# TrueMark

# CAPSTONE PROJECT PHASE-II

## Phase – II Report

*Submitted by*


1. **22BCE10354 Jit Surani**
2. **22BCE10364 Rushabh Wagh**
3. **22BCE10385 Kavya**
4. **22BCE10853 Tejas Pathak**
5. **22BCE10914 Simarpreet Singh**


*in partial fulfillment of the requirements for the degree of*

*Bachelor of Technology*

*in*

*Computer Science and Engineering*


VIT®
BHOPAL
www.vitbhopal.ac.in


**VIT Bhopal University**
**Madhya Pradhesh**


**March 2026**

# Bonafide Certificate

Certified that this project report titled **"TrueMark"** is the bonafide work of "22BCE10354 Jit Surani, 22BCE10364 Rushabh Madan Wagh, 22BCE10385 Kavya, 22BCE10853 Tejas Pathak, 22BCE10914 Simarpreet Singh**"** who carried out the project work under my supervision.

This project report (DSN4092-Capstone Project Phase-II) is submitted for the Project Viva-Voce examination held on **27.03.2026**

**Supervisor**
Dr. Sasmita Padhy
Program Chair, CSE-Core, SCOPE
VIT Bhopal University

# Table of Content

# List of Figures



# List of Tables

# Chapter 1: Introduction

## 1.1 Motivation

In the current digital era, the exchange of images and documents has increased exponentially across social media platforms, messaging services, and cloud-based systems. Users frequently share highly sensitive content such as identity proofs, academic records, medical documents, legal files, creative designs, and intellectual assets. Despite this widespread sharing, adequate security mechanisms are often missing, leaving both individuals and organizations vulnerable to threats such as data tampering, identity theft, unauthorized redistribution, privacy violations, and misuse by artificial intelligence systems.

Conventional methods used for protecting digital images are no longer sufficient to handle modern security challenges. Visible watermarks can be easily removed through cropping, filtering, or advanced editing tools. Similarly, metadata-based protection techniques fail to offer reliability, as metadata can be deleted or altered using commonly available software. Legal copyright enforcement processes are typically slow and reactive, making them ineffective in preventing damage in real time. Additionally, modern AI systems often scrape publicly available images without obtaining explicit consent from creators, leading to loss of ownership, attribution, and control over content.

The rapid advancement of generative AI and the availability of large-scale datasets have further intensified these concerns. Content creators often find their work being used in training datasets for commercial AI models without acknowledgment or compensation. Individuals may unknowingly have their personal images incorporated into facial recognition systems. Organizations also face risks of intellectual property theft as proprietary visual data is collected and reused without authorization. These issues highlight the urgent need for strong, automated, and tamper-resistant protection mechanisms that can ensure ownership, maintain privacy, verify authenticity, and prevent unauthorized usage across platforms.

TrueMark is developed to address these critical gaps by introducing a comprehensive framework for digital image protection. Instead of relying on external metadata or visible marks, the system embeds security directly within the image using cryptographic techniques and invisible watermarking. By integrating protection at the pixel level, TrueMark ensures that security remains intact regardless of how the image is shared, transferred, or modified.

The development of TrueMark is guided by three core principles. The first principle focuses on the protection of ownership and intellectual property, enabling creators such as photographers, designers, and artists to establish verifiable proof of authorship and maintain a clear record of ownership. The second principle emphasizes the security and privacy of sensitive visual data, ensuring that personal and confidential content remains protected during storage and transmission through strong encryption and integrity verification. The third principle promotes ethical and responsible AI usage by enabling the creation of machine-readable signals that can help AI systems recognize protected content and respect creator rights.

With the rise of deepfakes and advanced image manipulation technologies, visual content can no longer be assumed to be authentic. This has created a growing trust deficit in digital media, especially in domains such as journalism, legal systems, and online communication. TrueMark aims to mitigate this issue by providing a reliable mechanism for verifying the authenticity and origin of digital images.

From a usability perspective, many existing security solutions are complex, expensive, or require technical expertise, limiting their adoption among general users. TrueMark is designed as a mobile-first application that simplifies advanced cryptographic processes into an intuitive user experience. By doing so, it makes high-level security accessible to a broader audience, including non-technical users.

Therefore, TrueMark represents a necessary and timely solution to the evolving challenges in digital image security. It is designed as a practical, user-friendly, and robust system that addresses real-world problems affecting individuals, professionals, organizations, and the digital ecosystem as a whole.

## 1.2 Objective

The primary objective of TrueMark is to design and implement a secure system for image protection and verification by integrating cryptographic techniques, steganographic watermarking, and encryption within a cross-platform mobile application. The project aims to demonstrate that tamper-evident security mechanisms can be embedded directly into digital images while maintaining ease of use for non-technical users.

### 1.2.1 Major Objectives

**1. Develop a Secure Image Protection Pipeline**

The main technical objective is to build a multi-layered protection framework that combines multiple security approaches to ensure strong and reliable image protection. This includes embedding invisible watermarks using Least Significant Bit (LSB) techniques, generating unique image fingerprints through secure hashing algorithms such as SHA-256, and applying encryption using Advanced Encryption Standard (AES) with 256-bit keys to ensure data confidentiality. The system also incorporates structured payload design with headers, length indicators, encrypted content, and integrity checks using CRC32 to support accurate extraction and validation.

**2. Build a Cross-Platform Mobile Application**

The project aims to develop a fully functional mobile application using the Flutter framework that allows users to capture or select images, process them through the protection pipeline, and manage protected files efficiently. The application provides features such as unique image identification, real-time processing feedback, metadata tracking, and verification tools for detecting tampering and confirming authenticity.

### 3. Implement Backend Infrastructure

A scalable backend system is developed using Firebase services to support secure user authentication, data storage, and system operations. This includes managing user accounts, storing image metadata, generating unique identifiers through atomic operations, and maintaining structured records of image processing and verification activities.

### 4. Create a Verification System

The system includes a robust verification module that allows users to extract embedded data, validate integrity through checksum verification, confirm password-based access, and cross-check image information with stored database records. The module provides clear results indicating whether an image is authentic, modified, or unauthorized.

### 5. Establish AI Protection Foundation

The project lays the groundwork for future enhancements aimed at preventing unauthorized use of images in AI training systems. This includes embedding machine-readable signals, supporting standardized metadata formats, and enabling integration with external platforms through APIs for automated protection checks.

### 6. Ensure Usability and Accessibility

A key objective is to design a user-friendly interface that simplifies complex security processes. The application follows modern design principles, provides real-time feedback, supports multiple platforms, and ensures that both novice and advanced users can effectively use the system without difficulty.

### 7. Validate Security and Performance

The project includes systematic testing to evaluate watermark robustness, encryption strength, system efficiency, and verification accuracy. It also involves analyzing performance metrics such as processing time and resource usage, along with documenting system limitations and security considerations.

### 1.2.2 Expected Outcomes

At the completion of this project, TrueMark is expected to deliver a functional prototype capable of secure image protection and verification. The system will demonstrate the integration of multiple security techniques within a unified architecture, provide a scalable backend solution, and offer a user-friendly interface for practical use. Additionally, it will establish a foundation for future advancements such as digital signatures, blockchain integration, and AI-based protection mechanisms.

# Chapter 2: Existing Work / Literature Review

Recent work consistently shows that a layered approach combining strong encryption for confidentiality with verifiable signatures for authenticity most effectively prevents unauthorized reuse of images by AI systems and adversaries in the wild. Below are the findings that directly support TrueMark's design goal: encrypt user photos end-to-end so AI tools cannot ingest or learn from them, and bind each file to a cryptographic signature so any modification is immediately detected during verification, preserving user control over sharing and provenance in practical mobile workflows.

## 2.1 A Secure Method for Image Signaturing using SHA256, RSA and Advanced Encryption Standard

The paper proposes a three-stage digital signature scheme for images that combines SHA-256 hashing, RSA public-key cryptography, and AES symmetric encryption to create and distribute a compact signature stored as a binary file alongside the image. The process hashes both the image bytes and the filename, encrypts the concatenated 512-bit hash with RSA (2048-bit modulus), and then wraps the RSA output with AES (128-bit key), producing a 2048-bit ciphertext that can later be verified by decrypting (AES→RSA) and re-hashing the received image and filename to check equality for integrity and authenticity. Experiments with tampering (renaming, grayscale conversion, Gaussian blur, and single-pixel change) show the method flags manipulations reliably; even a one-bit change alters the SHA-256 digest, and reported PSNR values for blurred and one-pixel edits demonstrate detectable deviations, while grayscale changes are also identified despite PSNR not being applicable due to dimension changes in the input vector.

## 2.2 A Study on Digital Signatures with Privacy Factor

For an image-security application, the paper's guidance supports a clean separation of concerns: use a strong hash (e.g., SHA-256) over immutable image bytes plus a defined metadata subset, sign the digest with an application or user private key (RSA/ECDSA), and verify with the corresponding public key to ensure integrity and provenance, while handling confidentiality via symmetric encryption of the image for storage and transport when needed. It also points to operational best practices: manage keys via a lightweight PKI or embedded certificates, consider blockchain or append-only logs to timestamp signatures, and choose efficient algorithms (e.g., ECDSA over prime curves) to reduce latency on mobile clients without weakening security, aligning with the survey's performance and deployment recommendations.

## 2.3 A Comprehensive Survey on Methods for Image Integrity

This survey synthesizes methods for image integrity assurance across active and passive forensics, organizing the field by manipulation type (e.g., copy-move, splicing, resampling, double JPEG) and by evidence sources such as JPEG quantization traces, CFA/PRNU sensor fingerprints, and transform-domain statistical cues, as well as modern deep-learning detectors for forgeries and deepfakes. It reviews watermarking and zero-watermarking for active integrity, and details passive pipelines including keypoint features (SIFT, LBP), artifact analysis for compression history, and CNN-based localization, mapping strengths, failure modes, and benchmarks through metrics and datasets used in recent works.

## 2.4 A Cryptographic Framework for Secure Medical Imaging in Smart Healthcare Environments

The article proposes a hybrid cryptographic framework that combines AES-128 for fast medical-image encryption with RSA-1024 for secure key exchange, aiming to protect MRI and similar imaging data in smart healthcare systems. Experiments on a large MRI dataset show that encryption is efficient (0.5–2.9 seconds per image, with faster decryption) and achieves high throughput, especially in AES CTR and OCB modes. Security analyses including entropy, NPCR, PSNR, histograms, and correlation tests indicate strong resistance to statistical and differential attacks, with encrypted images showing near-ideal randomness and greater than 99.5% pixel-change rates. Overall, the study concludes that the scheme offers a practical balance of speed and confidentiality for telemedicine and digital-health workflows, though real-world deployment would still require consideration of stronger key sizes and broader system-level security.

## 2.5 Securing Medical Images for Mobile Health Systems Using a Combined Approach of Encryption and Steganography

The chapter presents a novel encryption scheme tailored for medical image protection in mobile-health (m-health) systems, combining a symmetric cipher for bulk image data with an asymmetric key system to secure key distribution. The authors design and implement their approach specifically for mobile devices and wireless transport of medical imaging, evaluating its performance in terms of encryption and decryption speed, image fidelity, and resilience to statistical and differential attacks. Their results show that the scheme provides sufficient protection of image confidentiality while achieving acceptable processing overhead for m-health contexts, thereby enabling secure transmission and storage of medical images on mobile platforms.

## 2.6 Privacy-Preserving Biometrics Image Encryption and Digital Signature Technique Using Arnold and ElGamal

The authors present a comprehensive cryptographic model for securing biometric images (face, palmprint and iris) that integrates encryption, signature and authentication: first the image is digitally signed by a dual-key scheme (using the ElGamal Digital Signature algorithm), then it undergoes scrambling via an enhanced three-dimensional version of the Arnold Transform, followed by further encryption of transform parameters using the ElGamal Encryption algorithm. The scheme was tested on multiple biometric datasets (including the FEI face set and IIT Delhi palmprint and iris sets) and compared with existing methods; results show good randomness, resistance to various attacks and suitability for storing and transmitting sensitive biometric image data while preserving integrity and authenticity.

## 2.7 The Evolution of Digital Signature Technologies in Mobile Devices

This 2024 review synthesizes how mobile platforms can implement robust digital signatures by combining a PKI-backed framework, hardware security (TEE), X.509 certificates with revocation, and secure transport (TLS) to protect keys and bind identities during signing and verification on-device. It contrasts signature construction styles including hash-and-sign with SHA-256, asymmetric schemes like RSA, ElGamal, and ECC, and biometric-assisted flows, and concludes that ECC offers better efficiency than RSA on constrained devices while SHA-

256 remains a reliable digest for integrity, with operational challenges centered on private key protection, jurisdictional legality, and secure key storage on mobile operating systems.

## 2.8 Secure Mobile Computing Authentication Utilizing Hash, Cryptography and Steganography Combination

The paper demonstrates a practical pattern for combining cryptographic integrity with concealed transport: compute a strong hash (preferably SHA-256), optionally encrypt it with AES, and bind it to an image via LSB or a more robust embedding method, enabling side-channel verification without exposing secrets in storage or transit. Although TrueMark focuses on encrypting user photos and attaching verifiable signatures, the steganographic-sidecar approach can be adapted for offline or bandwidth-constrained contexts where signatures or tokens need to travel with content; engineering lessons include avoiding plaintext secrets in databases, selecting SHA-256 for tamper sensitivity, using AES for confidentiality of embedded metadata, and validating usability via creation and verification timing alongside visual quality checks like PSNR when embedding marks.

## 2.9 Synthesis and Relevance to TrueMark

The surveyed literature collectively validates TrueMark's architectural decisions and provides empirical evidence for design choices implemented in this project. The work presented in Section 2.1 directly informs the planned integration of RSA-based digital signatures with SHA-256 hashing for image integrity verification, a feature currently in the development roadmap. The frameworks described in Sections 2.4 and 2.5 demonstrate that AES symmetric encryption is computationally viable for mobile medical imaging applications, supporting TrueMark's current implementation of AES-256-CBC for protecting user images on resource-constrained devices. The comprehensive forensics survey in Section 2.3 contextualizes TrueMark's LSB steganography within the broader field of image integrity methods, while the approach detailed in Section 2.8 provides practical validation for combining cryptographic hashing with steganographic embedding—the exact methodology TrueMark employs for invisible watermarking.

The biometric protection scheme described in Section 2.6 and the mobile signature evolution review in Section 2.7 establish that mobile platforms can support sophisticated cryptographic operations including ECC signatures and multi-stage encryption, affirming the feasibility of TrueMark's Flutter-based cross-platform architecture. These works demonstrate that computational overhead, key management complexity, and user experience considerations can be successfully addressed in mobile cryptographic applications.

Notably, no existing work surveyed combines steganographic watermarking, AES encryption, unique identifier generation, and Firebase-backed verification into a single user-friendly mobile application specifically designed for general-purpose image protection. Existing solutions either focus on specialized domains (medical imaging, biometric authentication) or implement only subsets of the required functionality (encryption-only or watermarking-only approaches). TrueMark addresses this gap by integrating multiple security layers—confidentiality through encryption, authenticity through unique identifiers, integrity through checksums, and ownership tracking through cloud-based verification—within an accessible mobile interface suitable for non-technical users.

Furthermore, the literature reveals a consistent trade-off between security strength and computational efficiency that TrueMark navigates through careful algorithm selection. While

papers surveyed employ varying key sizes (AES-128 vs AES-256, RSA-1024 vs RSA-2048), TrueMark's choice of AES-256-CBC with SHA-256 key derivation positions the system toward stronger security parameters while maintaining acceptable performance on modern mobile devices. The planned migration to AES-GCM and integration of elliptic curve digital signatures (as suggested by Section 2.7) will further enhance security properties while improving efficiency compared to traditional RSA approaches.

The convergence of findings across medical imaging security (Sections 2.4, 2.5), biometric protection (Section 2.6), and general-purpose mobile authentication (Sections 2.7, 2.8) validates TrueMark's hypothesis that layered cryptographic protection can be successfully deployed in mobile contexts without compromising usability. This literature foundation supports both the current LSB steganography implementation and the proposed evolution toward frequency-domain watermarking (DCT/DWT) and digital signature integration, representing TrueMark's novel contribution to practical image protection systems for everyday users.

## 2.10 Robust Image Watermarking Using Discrete Cosine Transform (DCT)

This paper presents a frequency-domain watermarking technique using the Discrete Cosine Transform (DCT) to improve robustness against common image processing operations. Unlike spatial-domain methods such as LSB, the watermark is embedded in mid-frequency DCT coefficients, balancing invisibility and resilience. The system encodes watermark bits into selected coefficients and reconstructs the image using inverse DCT, ensuring minimal perceptual distortion.

Experimental results demonstrate strong resistance against JPEG compression, noise addition, cropping, and filtering attacks. Performance evaluation using PSNR and Structural Similarity Index (SSIM) confirms that the watermark remains imperceptible while maintaining high recoverability after transformations.

This work is directly relevant to TrueMark’s future roadmap, where frequency-domain watermarking (DCT/DWT) is proposed to replace or enhance LSB-based embedding. It validates that moving to transform-domain techniques can significantly improve robustness in real-world scenarios such as social media compression and image sharing platforms.

## 2.11 Elliptic Curve Cryptography for Mobile Digital Signatures

This paper investigates the use of Elliptic Curve Cryptography (ECC) for efficient digital signatures in mobile environments. The authors propose an ECC-based signature scheme using smaller key sizes compared to RSA while maintaining equivalent security levels. The study evaluates performance metrics such as computation time, memory usage, and energy consumption on mobile devices.

Results show that ECC-based algorithms (such as ECDSA) significantly reduce computational overhead and power consumption while maintaining strong security guarantees. The reduced key size also lowers bandwidth requirements during transmission, making it ideal for mobile and IoT applications.

For TrueMark, this paper supports the planned transition from RSA-based signatures to ECC-based mechanisms, improving performance and scalability in mobile environments without compromising cryptographic strength.

## 2.12 Deep Learning-Based Image Forgery Detection Using Convolutional Neural Networks

This study explores the application of Convolutional Neural Networks (CNNs) for detecting image forgeries and deepfake manipulations. The model is trained on large datasets containing tampered and authentic images, learning subtle artifacts introduced during editing processes such as splicing, retouching, and GAN-based synthesis.

The proposed CNN architecture achieves high accuracy in detecting manipulated regions and classifying images as authentic or forged. The system outperforms traditional feature-based methods by automatically learning hierarchical features without manual extraction.

This research aligns with TrueMark's future enhancement of integrating AI-based deepfake detection. It demonstrates that combining cryptographic verification (active methods) with AI-based forensic detection (passive methods) can create a comprehensive image authenticity framework.

## 2.13 Secure Key Management Using Hardware-Based Trusted Execution Environments

This paper examines secure key storage and cryptographic operations using hardware-backed Trusted Execution Environments (TEE) available in modern mobile devices. It highlights implementations such as secure enclaves and keystore systems that isolate sensitive cryptographic material from the main operating system.

The study demonstrates that storing private keys within a TEE significantly reduces the risk of key extraction attacks, even in compromised software environments. Performance evaluations show minimal latency overhead for cryptographic operations, making it practical for real-time applications.

This directly supports TrueMark's use of OS-level secure storage mechanisms for PIN and key management. It validates the design choice of leveraging hardware-backed security (Android Keystore / iOS Keychain) to ensure strong protection of user credentials and cryptographic secrets.

## 2.14 Hybrid Cryptography for Secure Multimedia Transmission

The paper proposes a hybrid cryptographic model combining symmetric encryption (AES) for bulk data and asymmetric encryption (RSA/ECC) for secure key exchange. The system is designed for secure multimedia transmission over untrusted networks, ensuring confidentiality, integrity, and authentication.

Experimental results indicate that hybrid encryption significantly improves performance compared to purely asymmetric systems, while maintaining strong security properties. The approach also minimizes latency during transmission, making it suitable for real-time applications.

This work reinforces TrueMark's TrueLock module design, which uses symmetric encryption for file protection and password-derived keys for secure access. It validates the efficiency and practicality of hybrid cryptographic approaches in mobile environments.

## 2.15 Zero-Watermarking Techniques for Image Authentication

This paper introduces zero-watermarking, a technique where no actual data is embedded into the image. Instead, unique features are extracted from the image and combined with a watermark to generate a signature stored externally. During verification, the same features are re-extracted and matched with the stored signature.

The approach ensures that the original image remains completely unaltered, avoiding any perceptual distortion. It also provides robustness against attacks such as compression and filtering since the watermark is not embedded directly.

For TrueMark, this presents an alternative approach to watermarking that could complement or replace steganographic embedding in sensitive use cases. It highlights the possibility of achieving authenticity without modifying the original media.

## 2.16 Blockchain-Based Image Provenance and Copyright Protection

This paper explores the use of blockchain technology for maintaining immutable records of image ownership and provenance. Each image is hashed using SHA-256 and stored on a distributed ledger along with ownership metadata and timestamps.

The decentralized nature of blockchain ensures that records cannot be altered or deleted, providing strong non-repudiation and transparency. Experimental implementations demonstrate feasibility with acceptable transaction costs and latency.

This directly aligns with TrueMark's future roadmap of transitioning from centralized Firebase storage to blockchain-based verification. It validates the feasibility of decentralized ownership tracking and strengthens the system's trust model.

## 2.17 Performance Analysis of AES Modes for Secure Image Encryption

This paper compares different modes of Advanced Encryption Standard (AES), including CBC, CTR, and GCM, for image encryption. The evaluation considers factors such as encryption speed, security properties, and resistance to attacks.

Results show that AES-GCM provides both confidentiality and authentication, making it superior to AES-CBC, which only ensures confidentiality. CTR mode offers high speed but lacks built-in integrity protection.

This study supports TrueMark's planned upgrade from AES-CBC to AES-GCM, highlighting the importance of authenticated encryption in preventing tampering during storage or transmission.

## 2.18 Steganalysis Techniques for Detecting LSB-Based Hidden Data

This paper investigates methods for detecting hidden data in images using statistical steganalysis techniques. It analyzes pixel distributions, noise patterns, and histogram anomalies to identify the presence of LSB-based steganography.

The results indicate that while LSB methods are simple and efficient, they are vulnerable to detection under advanced statistical analysis. Machine learning-based steganalysis further improves detection accuracy.

This work provides critical insight into the limitations of TrueMark's current LSB-based TrueHide module. It justifies the need for more advanced embedding techniques or encryption layering to maintain security against adversarial analysis.

## 2.19 Synthesis and Extended Relevance to TrueMark

The additional literature further strengthens the theoretical foundation of TrueMark by expanding across three key dimensions: robustness, efficiency, and trust decentralization. The studies on DCT-based watermarking (Section 2.10) and steganalysis (Section 2.18) highlight both the strengths and limitations of spatial-domain embedding, reinforcing the need for TrueMark's planned transition to frequency-domain techniques.

The works on ECC (Section 2.11) and AES mode comparison (Section 2.17) provide strong justification for optimizing cryptographic performance and adopting authenticated encryption, ensuring both efficiency and security in mobile environments. Meanwhile, blockchain-based provenance systems (Section 2.16) validate the long-term vision of decentralized verification, addressing the limitations of centralized trust models.

Additionally, research on deep learning-based forgery detection (Section 2.12) introduces complementary passive verification techniques that can enhance TrueMark's active cryptographic approach. The combination of these methods supports a hybrid framework where authenticity is verified through both embedded signatures and AI-driven analysis.

Overall, the extended literature confirms that TrueMark's layered architecture—combining encryption, watermarking, and verification—aligns with state-of-the-art research trends. It also highlights clear pathways for future improvements, positioning TrueMark as a comprehensive and scalable solution for secure image protection and digital authenticity in modern mobile ecosystems.

# Chapter 3: Front End, Backend and System Requirement

## 3.1 System Architecture Overview

TrueMark is a cross-platform mobile application developed using the Flutter framework, implementing a client-server architecture with Firebase as the backend service provider. The application focuses on digital image watermarking and verification using steganographic techniques combined with cryptographic encryption. The system follows a three-tier architecture: Flutter-based frontend for user interaction, Firebase Backend-as-a-Service (BaaS) for authentication and data management, and a local cryptographic services layer for steganography and encryption operations.

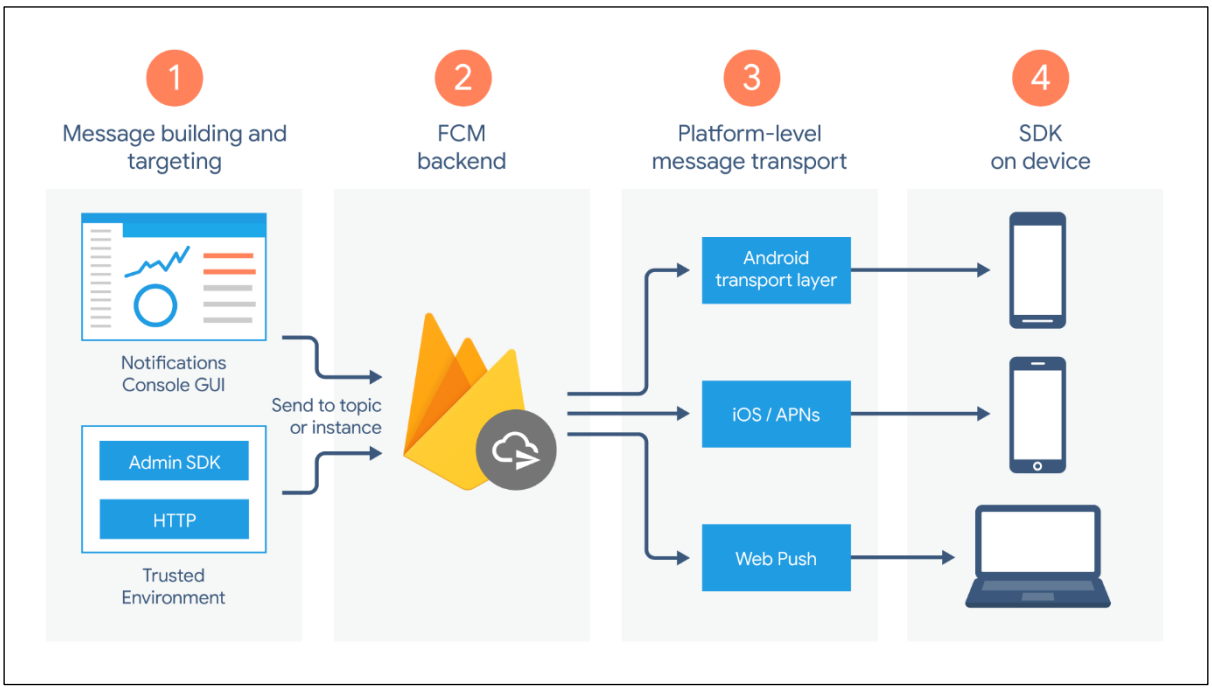



**Figure 1 – System Architecture Overview**

## 3.2 Frontend Requirements

### 3.2.1 Framework and Technology Stack

Primary Framework:

- Flutter SDK: Version 3.8.1 or higher
- Dart Programming Language: Version 3.8.1 or higher
- Material Design 3 UI components with Indigo-based theme

Key Frontend Dependencies:

Core UI Libraries:

- flutter (SDK): Core application framework
- cupertino_icons (v1.0.8): iOS-style icons
- google_fonts (v6.1.0): Custom typography integration
- oktoast (v3.3.1): Toast notifications for user feedback

Media Handling:

- image_picker (v0.8.7+5): Camera and gallery access
- image (v4.5.4): Image decoding and pixel-level processing
- flutter_image_compress (v2.3.0): Image resizing and optimization (e.g., 1080p constraint for steganography)
- path_provider (v2.0.14): Access to device-specific file storage paths

Cryptography Integration:

- crypto (v3.0.2): SHA-256 hashing for content fingerprinting
- encrypt (v5.0.3): AES encryption/decryption APIs
- pointycastle (v3.9.1): Low-level cryptographic implementations

Utilities:

- path (v1.8.2): File path manipulation
- device_info_plus (v10.0.2): Device-level information access
- intl (v0.18.1): Formatting and localization support

### 3.2.2 Screen Components

The application consists of structured interface screens aligned with the TrueMark workflow system:

1. Signup Screen: User registration with email/password, validation, and Firebase Authentication integration
2. Login Screen: Secure authentication and session initialization
3. Profile Setup Screen: Captures user identity (name and username) for ownership mapping
4. Home Screen (Workflow Hub): Central dashboard providing access to core modules (TrueSign, TrueLock, TrueHide, TrueMeta) with guided user actions
5. TrueSign Interface: Handles file/image selection, signature embedding, and ownership registration workflows
6. Verification Screen: Extracts embedded signatures, performs decryption, and validates ownership against Firestore registry

7. TrueLock Interface: Manages file encryption and decryption into/from .tmk containers
8. TrueHide Interface: Supports payload embedding and extraction with capacity validation and controlled processing
9. TrueMeta Interface**:** Displays metadata inspection results and enables sanitization operations
10. TrueVault Screen: Provides secure access to stored artifacts with filtering, sorting, and categorized views
11. Processing/Status Screens: Display real-time progress for operations such as encryption, embedding, verification, and metadata cleaning

### 3.2.3 Frontend Features

Navigation & Workflow Design:

- Structured, workflow-driven navigation guiding users through inspection → protection → verification → storage
- Route management using push and pushReplacement for authentication and feature transitions
- Modular access to each security component through a centralized dashboard

User Interaction:

- Real-time form validation and error handling for authentication and input fields
- Dynamic loading indicators during asynchronous operations (encryption, embedding, verification, sanitization)
- Toast notifications for system feedback (success, failure, warnings)
- File/image preview with metadata (name, size, type) prior to processing

State Management:

- StatefulWidget and setState for UI state handling
- Firebase Authentication state tracking via currentUser
- Integration with backend services (Firestore, REST fallback on Windows) for dynamic updates

Responsive & Cross-Platform Design:

- Scrollable layouts using SingleChildScrollView for overflow handling
- Constrained layouts ensuring consistent UI across devices and desktop environments
- Adaptive design supporting Android and Windows platforms
- Platform-aware behavior (e.g., file handling differences, Firestore REST fallback on Windows)

Data Handling & Safety:

- Controlled file processing pipelines to prevent partial or corrupted operations
- Automatic image resizing (e.g., 1080p constraint) to ensure predictable steganographic capacity
- Secure local storage integration for TrueVault with isolated file access

## 3.3 Backend Requirements

### 3.3.1 Firebase Backend-as-a-Service

Firebase Project Configuration:

- Project ID: truemark-5f8bb
- Platform Support: Android, Windows (with Web/iOS/macOS compatibility)
- Configuration File: lib/firebase_options.dart with platform-specific API keys

Required Firebase Services:

*Firebase Authentication:*

- Email/password-based authentication
- Session persistence with automatic token refresh
- User identity linked to ownership records

*Cloud Firestore:*

- NoSQL document database used as a centralized ownership registry
- On-demand queries for verification (no real-time listeners used)
- Document-based lookup using SHA-256 hash as unique identifier
- REST API fallback implemented for Windows platform compatibility

*Firebase Storage:*

- Provisioned but not actively used in the current implementation
- Reserved for future cloud backup and media storage features

### 3.3.2 Database Schema

Firestore Collections:

*users/{uid} — User profile documents:*

- email (string): User email address
- name (string): User name

- username (string): Unique identifier for ownership mapping
- createdAt (timestamp): Account creation time

*ownership/{hash} — Ownership registry:*

- hash (string): SHA-256 hash of file/image (document ID)
- uid (string): Owner's user ID
- username (string): Owner's username
- timestamp (string): Signature creation time
- metadata (map): Optional additional details

Note:

- SHA-256 hash is used as the primary key for fast and deterministic verification
- Eliminates need for sequential ID generation systems

*login_attempts (optional/stubbed):*

- Structure exists for authentication logging
- Logging functionality currently not active (no-op in implementation)

### 3.3.3 Backend Services Layer

TrueSign Service:

- Generates SHA-256 hash of content
- Constructs signature payload (UID | timestamp | hash)
- Encrypts payload and embeds into media using LSB steganography
- Registers ownership record in Firestore
- Verifies extracted signatures against registry

TrueLock Service:

- Encrypts files into .tmk container using AES-256-CBC with PKCS7 padding
- Generates random IV for each encryption
- Stores metadata (file extension, header signature, IV, ciphertext)
- Supports deterministic decryption restoring original file

TrueHide Service:

- Embeds encrypted payloads (text or binary) into images using LSB steganography
- Performs capacity estimation before embedding
- Applies automatic image resizing (e.g., 1080p constraint)
- Supports secure extraction using shared key

**TrueMeta Service:**

- Extracts EXIF and media metadata for analysis
- Categorizes metadata (device, location, timestamps)
- Performs metadata sanitization via decode/re-encode process
- Outputs clean media files with no residual metadata

**TrueVault Service:**

- Manages secure local storage of generated artifacts
- Enforces PIN-based access control using secure storage
- Supports categorized file access and persistent user preferences
- Ensures atomic file operations to prevent corruption

## 3.4 System Requirements

### 3.4.1 Development Environment

Core Tools:

- Flutter SDK ≥ 3.8.1
- Dart SDK ≥ 3.8.1
- Git for version control
- IDE: Visual Studio Code / Android Studio / IntelliJ IDEA

Platform-Specific Development:

- Android: Android Studio, Android SDK (API 21+), JDK 11+
- Windows: Visual Studio 2019+ with C++ build tools, Windows 10 SDK

Firebase Tools:

- Firebase CLI
- FlutterFire CLI for configuration
- Active Firebase project with Authentication and Firestore enabled

### 3.4.2 Runtime Requirements

Mobile (Android):

- Android 5.0+ (API 21 or higher)
- Minimum 2GB RAM

- ~100MB storage for application
- Camera and internet connectivity required

Desktop (Windows):

- Windows 10 or higher
- Minimum 4GB RAM
- ~200MB storage
- Stable internet connectivity

Note:

- System optimized for Android and Windows platforms
- Other platforms are theoretically supported by Flutter but not fully validated

### 3.4.3 Performance Specifications

Processing Performance:

- Image processing (embedding, extraction, encryption): Typically a few seconds depending on image size and device capability
- Metadata analysis and sanitization: Near-instant for standard images
- Firestore operations: Sub-second response under stable network conditions

Resource Usage:

- Memory usage may temporarily increase (up to ~2–3× file size) during image processing
- Automatic resizing (e.g., 1080p constraint) applied to prevent excessive memory usage and ensure predictable processing

Storage Requirements:

- Application size: ~100–120MB
- Temporary processing files may require additional space
- Recommended free storage: Minimum 500MB

Network Requirements:

- Minimum: 1 Mbps connection
- Recommended: 5 Mbps for smooth operation
- Required for authentication and ownership verification

### 3.4.4 Security and Permissions

Required Permissions (Android):

- INTERNET (for Firebase services)
- CAMERA (for image capture)
- Media/file access via scoped storage

Data Protection:

- All network communication secured via HTTPS (TLS)
- Authentication tokens stored securely using platform mechanisms
- Sensitive operations performed locally (encryption, embedding, sanitization)
- Ownership data stored in Firestore with controlled access rules

### 3.4.5 Platform-Specific Configurations

Android:

- Minimum SDK: 21
- Target SDK: Latest stable
- Firebase configuration via google-services.json
- Gradle and Kotlin configured for Flutter compatibility

Windows:

- CMake-based build system
- Fixed window constraints for consistent UI behavior
- Firestore access via REST API fallback for compatibility

Cross-Platform Notes:

- File handling and storage paths adapted per platform
- UI optimized for both mobile and desktop form factors
- Backend interactions abstracted to support platform-specific constraints

## 3.5 Summary

TrueMark's architecture integrates Flutter's cross-platform capabilities with Firebase's serverless backend to deliver a comprehensive digital forensics and privacy suite. The frontend provides a structured, workflow-driven interface enabling users to perform media inspection, ownership embedding, encryption, verification, and secure storage within a unified environment, primarily optimized for Android and Windows platforms.

The backend leverages Firebase Authentication for user identity management and Cloud Firestore as a centralized ownership registry, where records are indexed using SHA-256 hashes for fast and deterministic verification. On Windows, a REST API fallback is implemented to ensure compatibility with Firebase services. Unlike earlier designs, the system avoids sequential ID generation and instead relies on content-based hashing for efficient lookup and integrity validation.

Core security operations are performed client-side to preserve privacy and minimize data exposure. These include cryptographic processing (AES-256-CBC encryption with PKCS7 padding), steganographic embedding and extraction using LSB techniques, and metadata analysis and sanitization. The system follows a layered security approach combining cryptography, steganography, and registry-based verification to ensure authenticity, confidentiality, and operational safety.

System requirements are moderate, supporting devices running Android 5.0 or higher and Windows 10 or above, with minimum memory requirements of approximately 2GB for mobile and 4GB for desktop environments. Processing performance is dependent on device capability and file size, typically completing operations such as embedding, encryption, and verification within a few seconds. Memory usage is managed through controlled processing techniques, including image resizing constraints to ensure stability.

The modular design of TrueMark enables future extensibility, including enhancements such as AI-based deepfake detection, advanced watermarking techniques, and decentralized verification mechanisms. Overall, the system demonstrates a practical and scalable approach to addressing modern challenges in digital authenticity, security, and privacy.

# Chapter 4: Methodology / Topic of work

This chapter outlines the general structure and operating process of TrueMark, a multi-layer network digital security and forensic system construction that was aimed at providing authenticity, privacy, and data security. The modules that are currently being implemented are LSB-based steganographic signature embedding, AES-256-GCM encryption with PBKDF2 key derivation, metadata analysis and sanitization, secure local storage, and a Firestore-based verification registry.

All the modules collaborate to offer a stratified method of digital trust, which is a combination of cryptographic methods, data hiding, and forensic examination. The system will be strengthened by further additions in future such as verification based on blockchain, deep fake detection, and biometric authentication, which will be implemented later on.

## 4.1 System Design / Architecture

TrueMark is a complete secure media protection system that adds trust directly to digital images before sharing. It uses a layered security model that combines invisible watermarking, cryptographic signatures, and encryption. This approach ensures confidentiality, authenticity, and integrity during the image lifecycle.

1. Core Architectural Principles
   - Layered Security: There are various protection mechanisms (encryption, hashing, steganography) utilized that mitigate a failure point.
   - Modularity: It contains modules that act independently (TrueSign, TrueLock, TrueHide, TrueMeta, True Vault), which allows scaling and allowance of easy maintenance.
   - Zero-Trust Technique: This is an encryption and validation approach where all of the operations (encryption, validation, access) are required to be validated and nobody is trusted needlessly (which includes the data or the users).
   - Integrity First: SHA-256 Hashing There is no way to change the data without it being noticed since it will undergo a hashing algorithm of SHA-256.
   - Privacy by Design: Metadata sanitization/secure data storage are normal features, not an optional feature.

2. Layered Protection Pipeline

   TrueMark operates on a step-by-step pipeline whereby file goes through numerous levels of security before it is ultimately either stored or shared.

   - Input Stage: The user picks a file either in the device or TrueVault.
   - Processing Stage: AES-256-GCM is used to encrypt the file, the keys are generated through PBKDF2. Optional or steganographic embedding (LSB) can be used.
   - Verification Stage: SHA-256 hash will be generated to develop a distinct fingerprint which will be hit in Firestore registry to verify integrity and ownership.

- Privacy Stage: Stage performs a scan and destroys sensitive system metadata: GPS position, data about the device, and EXIF data.
- Storage Stage: The last file is safely stored in the TrueVault which is secured using OS-level security and PIN authentication.

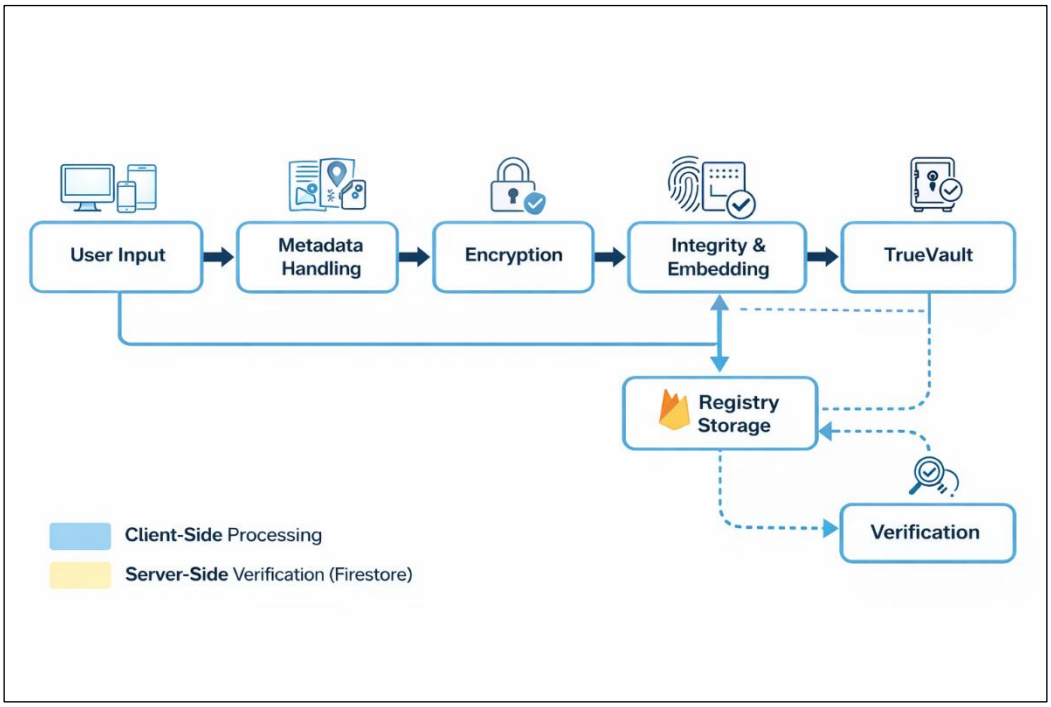



**Figure 2 – System Design**

3. Key Components

| Component | Function |
|---|---|
| TrueSign | Embeds and authenticates with the help of SHA-256 hashing and LSB steganography with registry validation. |
| TrueLock | File encryption with keys based on PBKDF2 with AES-256-GCM. |
| TrueHide | Hides confidential information in pictures with the help of steganography based on LSB. |
| TrueMeta | Extracts and removes metadata such as EXIF, GPS, and device identifiers |
| TrueVault | Metadata + encrypted file storage |

**Table 1 - TrueMark Components**

4. Secure Data Flow

- User Input: This begins at the point where the user finds the file whether it is on their device or on the secure TrueVault.
- Pre-Processing: The system initially examines the file with regards to the existence of metadata and is able to clean sensitive data.
- Security Processing: AES-256-GCM is used to encrypt the file to ensure the contents of the file are secure. A hash of SHA-256 is made in order to come with a distinct identity of the file. Hidden data or signatures may be carried with the help of steganography in case it is necessary.
- Registry Interaction: In order to make the hash and ownership information valid, the hash and the ownership generated is stored or checked through the Firestore database.
- Storage/Output: Lastly the file is stored in TrueVault or it is exported into the participants machine.

This traffic keeps all files safe, original, and free of sensitive information prior to data storage or sharing.

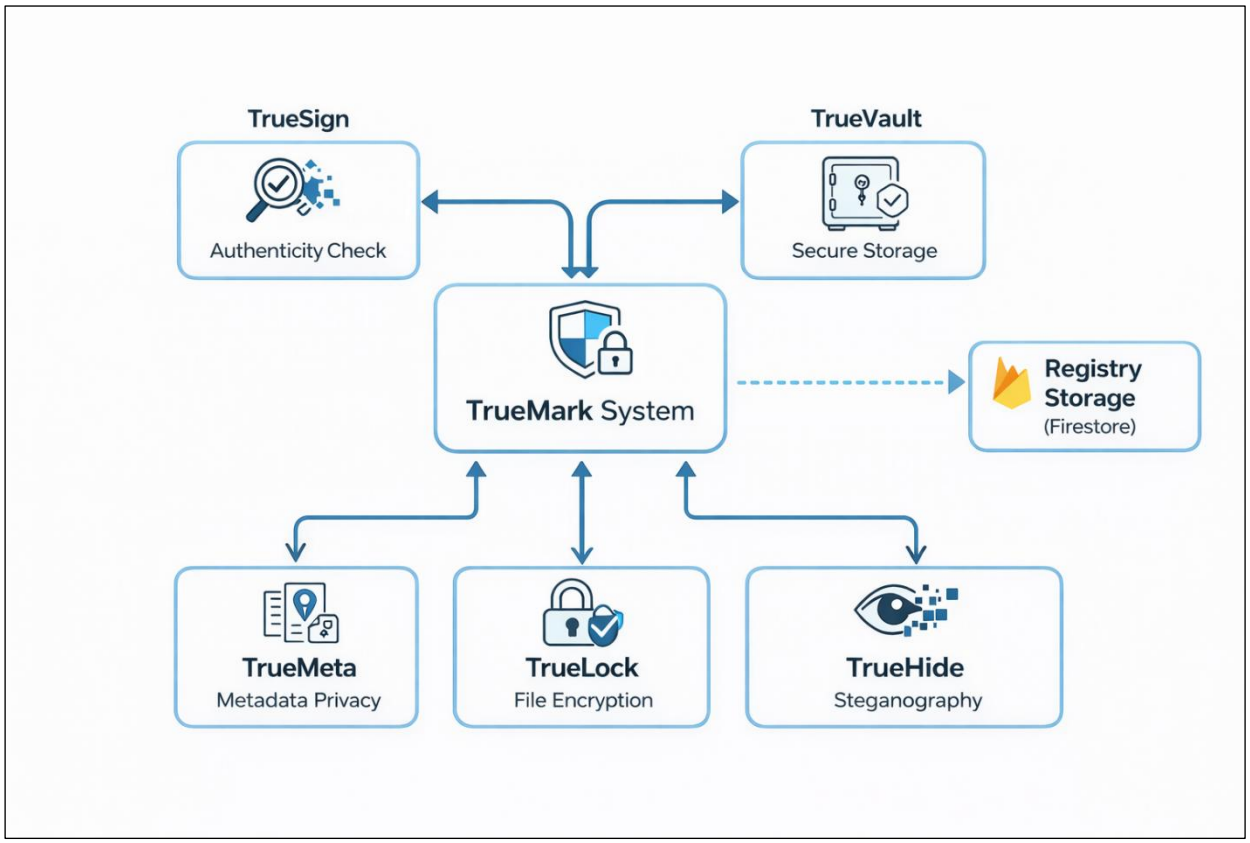



**Figure 3 – TrueMark Flow**

## 4.2 Working Principle

The Working Principle can be simplified into two processes to comprehend the operations of TrueMark: client-side processing and server-side verification.

- Client-side It is conducted at the local level in the device of the user in order to execute sensitive operations like encryption, hashing, steganography and metadata sanitization to achieve privacy and security.
- Firebase fire is a server implemented as a verification registry on the server side, into which file hashes and ownership information is stored and retrieved to ensure authenticity.

This isolation is such that raw data does not leave the user computer and only the necessary verification data is processed on the cloud.

### 4.2.1 End to End flow

The entire justice of TrueMark can be outlined in the following way:

- User selects a image
- Sanitization and optional metadata analysis is done.
- The encryption of the file is based on AES-256-GCM.
- Integrity is achieved by generating SHA-256 hash. Here, signature or concealed data is attached with LSB (where necessary).
- Hash and ownership are stored in Firestore
- File can be kept securely at TrueVault or saved.
- In the process of verification, the embedded information is removed and compared against records in the registry.

### 4.2.2 Detailed Working Principle

- Step 1 – File Selection
  - User captures or imports an image via the mobile application.
  - The system prepares the file to be further processed.
- Step 2 – Metadata Management
  - Metadata secrets in the file, including EXIF, gps, device, etc., are scanned and sensitive data deleted to safeguard the privacy of the user.
- Step 3 – Encryption
  - The file will be encrypted with AES-256-GCM
  - Strong key will be created with the help of PBKDF2 depending on the input of the user.
- Step 4 – Integrity & Data Embedding
  - A hash value using the SHA-256 algorithm is taken to be used to identify file uniquely
  - If it is needed, a signature or hidden data can be embedded using LSB method.
- Step 5 – Registry Storage
  - The hash and ownership information is stored in Firestore

    - This allows one to verify and track authenticity later
- Step 6 – Secure Storage / Output
    - Image can be exported or saved to device
- Step 7 – Verification Process
    - To verify it, the embedded data is read out of the file
    - The hash generated or the existing one is compared to Firestore records in order to verify authenticity.

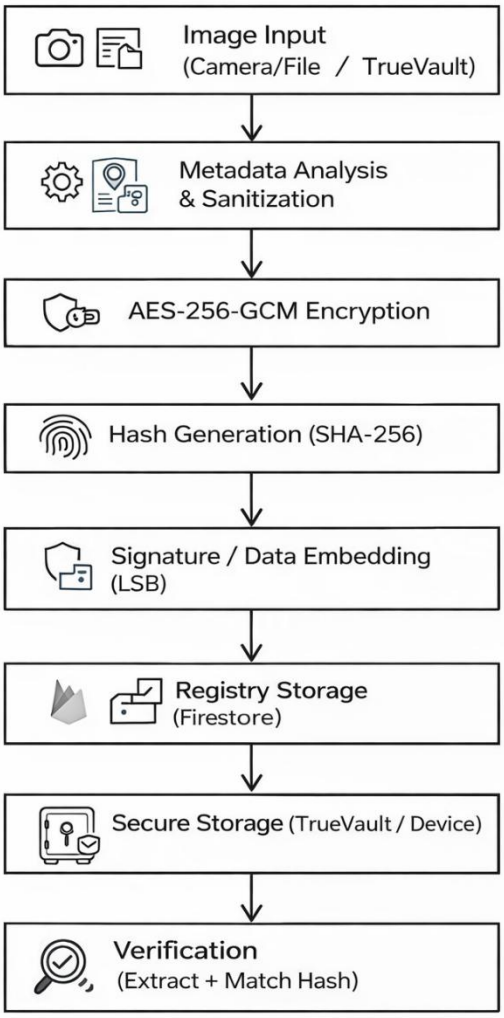



**Figure 4 – End to End Flow**

### 4.2.3 Verification Process

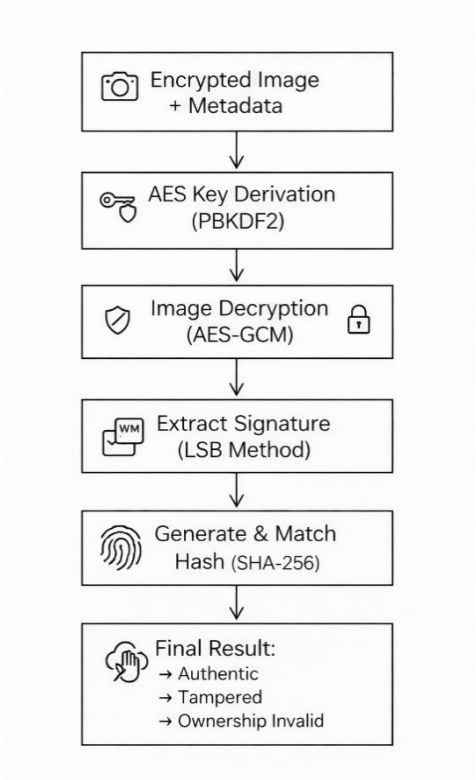



**Figure 5 – Verification Process**

### 4.2.4 Decision Flow

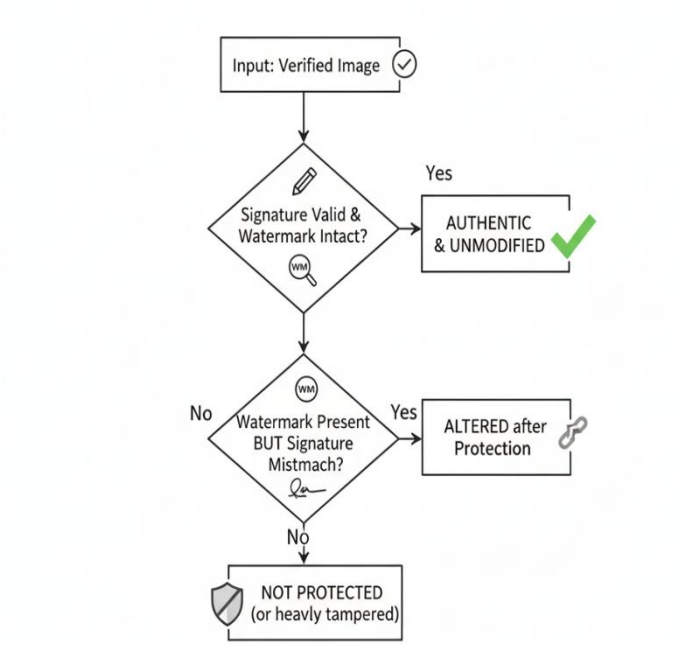



**Figure 6 – Decision Flow**

## 4.3 Results and Discussion

The section shows the results of the present TrueMark implementation, its drawbacks, and its viability as well as the comparison with the existing methods. The idea is to determine the degree to which the system is capable of meeting the digital authenticity, security, and privacy issues.

### 4.3.1 Implementation Results

The following is currently under implementation and it shows a successful multi-layer security system in place:

- The system can also readily encrypt and decrypt files with integrity and confidence using the AES-256 -GCM encryption system.
- SHA-256 hashing is quite effective to indicate any alteration in files.
- LSB steganography enables the integrity of steganographic extraction and steganographic insertion of concealed information or signature in images.
- Firestore registry allows having fundamental ownership tracking and file verification. Intelligent metadata analysis tool is effective in determining and eliminating sensitive data like GPS and device information.
- TrueVault offers a local secure storage setting that has limited access.

### 4.3.2 Current Implementation Limitations

Even being functional, there are certain limitations of the current system:

- LSB steganography is weak.
- It will resolve in the failure case when the image is compressed, resized, or changed.
- It is not entirely immune to more sophisticated forensic detection.
- Firestore registry Centralized (Firestore) Bases on a faith in a top-down system.
- It's not immutable completely
- Performance constraints
    - Encryption and processing of large files can be cumbersome in low-end computers.
- Security trade-offs
    - PBKDF2 is safe and not as powerful as new such as Argon2.
    - The access to vaults via PIN is less than when utilizing biometrics.

Steganographic Robustness Issues:

- JPEG Compressional Failure: LSB Spatial Domain Watermark is unable to withstand JPEG compressional algorithms; social media sites that re-compress JPEG images completely destroy embedded watermarks.

- Rescaling Vulnerability: LSB Watermarks are unable to withstand rescaling, and any change in image size via interpolation causes failure

- Format Conversion Vulnerability: Only PNG is a lossless image format that is able to withstand conversion and maintain embedded watermarks; JPEG, WebP, and other formats completely destroy embedded watermarks
- Single-Channel Vulnerability: Using only one color channel, such as blue, in LSB is a huge weakness, and any alteration of a single channel causes failure
- No Error Correction: There is no error correction in LSB, and any corruption of a single bit causes complete failure

Operational Constraints:

- Local-only storage: Images are stored in temporary directories on the local device, and there is no backup to the cloud; loss of the device results in loss of data
- Manual password management: User needs to remember unique IDs for each image; there is no mechanism to recover passwords
- Single-image workflow: No support for batch processing; images are sent one at a time, and the entire UI flow is required to be completed for one image at a time
- Platform limitation: Currently, the application is only for Android; cross-platform support (for iOS, Windows, macOS, and Web) is not yet implemented

### 4.3.3 Feasibility Analysis of Proposed Architecture

The proposed system architecture is also feasible in the following ways:

- Technically feasible
    - The proposed system has been built using Flutter and Firebase, hence it is cross-platform and scalable.
    - It has also adopted widely known cryptographic standards. Therefore, the proposed system is technically feasible.

- Operationally feasible
    - The proposed system has been designed to operate in a manner that minimizes privacy risks. This has been achieved by limiting the operations to local processing. Moreover, the use of cloud technology has been limited to verification data. Therefore, the proposed system is operationally feasible.

- Scalability
    - The proposed system has also been designed to be scalable. This has been achieved by using a modular design.

### 4.3.4 Comparative Analysis

| Feature | Traditional Tools | TrueMark |
|---|---|---|
| Encryption | Basic AES (often without auth) | AES-256-GCM (confidentiality + integrity) |
| Data Hiding | Rare or separate tools | Integrated steganography (LSB) |
| Integrity Check | Limited | SHA-256 based verification |
| Metadata Handling | Often ignored | Deep analysis and sanitization |
| Ownership Verification | Not available | Firestore-based registry |
| Secure Storage | Basic file storage | TrueVault (sandboxed storage) |

**Table 2 – Comparative Analysis**

### 4.3.5 Conclusion

TrueMark achieves a successful demonstration of a multi-layered approach to digital security by integrating encryption, steganography, hashing, and forensic analysis in a single system. It is capable of ensuring data confidentiality, detecting any possible tampering, and ensuring user privacy while still providing basic verification of ownership.

While there are certain limitations in steganography's robustness and centralized verification, overall, the system's architecture is sound and provides a good basis for future developments and improvements. With future developments such as blockchain and AI, TrueMark is a system with great potential to become a digital trust system in the future.

# Chapter 5: Individual Contribution by Members

**Name: Jit Nipulkumar Surani**

**Email:** jitnipulkumarsurani2022@vitbhopal.ac.in

**Project Role:** Project Lead, System Architect & Cryptography Specialist

**Contribution:** Jit served as the project lead responsible for overall coordination, planning, and architectural design of the TrueMark system. He designed and implemented the complete cryptographic protection pipeline, including AES-256-GCM authenticated encryption/decryption with PBKDF2-SHA256 key derivation and SHA-256 hashing for content integrity. He developed the LSB steganography service for invisible signature embedding and extraction within the TrueSign module, integrated with a Cloud Firestore-based verification registry for identity-linked validation. He also contributed to backend development, including secure data handling workflows and integration of cryptographic operations with database services. He conducted comprehensive security analysis identifying vulnerabilities and implemented improvements such as randomized IVs and strengthened key derivation. Additionally, he architected the modular layered security model and authored the conclusion and future scope synthesizing system outcomes and advancements.

---

**Name: Rushabh Madan Wagh**

**Email:** rushabhmadanwagh2022@vitbhopal.ac.in

**Project Role:** Backend Services Developer, Frontend Integration & Methodology Analyst

**Contribution:** Rushabh developed the backend service components supporting the TrueMark system, including Firestore-based registry operations for secure storage and retrieval of verification records. He implemented SHA-256 hash computation for content fingerprinting and designed metadata handling workflows integrated with the TrueSign verification pipeline. He contributed to frontend development in collaboration with Kavya, implementing the Home and Verification screens with file/image selection, processing workflows, and real-time feedback. He developed the integration layer connecting frontend components with backend services and cryptographic modules, ensuring seamless data flow between user interactions, Firebase operations, and encryption/steganography processes. Additionally, he authored the Methodology and Results sections detailing system architecture, implementation, outcomes, and feasibility analysis.

**Name: Kavya**

**Email:** kavya2022@vitbhopal.ac.in

**Project Role:** Frontend Developer & UI/UX Designer

**Contribution:** Kavya played a key role in the conceptualization and design of the TrueMark system, contributing to the idea of developing an integrated digital security suite and shaping the overall feature set and workflow architecture. He primarily designed and implemented the frontend of the application, including authentication interfaces such as Signup, Login, and Profile screens with form validation, real-time feedback, and Firebase Authentication integration. He developed core user interface flows for file selection, processing, and verification result display, ensuring a smooth and intuitive user experience. He established a consistent Material Design 3-based design system with an Indigo-themed color scheme, Google Fonts typography, and responsive layouts. He also implemented navigation systems, toast notifications, and dynamic feedback mechanisms enhancing usability across the application.

---

**Name: Tejas Pathak**

**Email:** tejaspathak2022@vitbhopal.ac.in

**Project Role:** Cloud Infrastructure Specialist & Motivation Analyst

**Contribution:** Tejas established and configured the complete Firebase backend infrastructure for the TrueMark system, including Firebase Authentication for secure user identity management and Cloud Firestore for the centralized verification registry and application data storage. He designed Firestore database structures supporting user records, verification entries, and system metadata, ensuring efficient and scalable data handling. He contributed to backend development by implementing secure access control through Firestore security rules, enforcing user-specific data isolation and protected operations. He configured platform-specific Firebase integration across Android, iOS, Web, Windows, and macOS environments, enabling seamless cross-platform functionality. Additionally, he authored the Introduction section, articulating project motivation and defining the overall objectives and scope of the system.

**Name: Simarpreet Singh**

**Email:** simarpreetsingh2022@vitbhopal.ac.in

**Project Role:** Research Analyst, QA Engineer & Literature Reviewer

**Contribution:** Simarpreet conducted a comprehensive literature review analyzing research in digital forensics, steganography, cryptography, and secure mobile systems, synthesizing findings that informed TrueMark's modular architecture and validated its layered security approach. He contributed to backend-related evaluation and testing by analyzing data flows, system performance, and integration stability across modules. He designed and executed testing across devices, evaluating encryption/decryption performance, embedding and extraction efficiency, and overall system reliability. He performed platform-level testing on Android, including permissions, build configuration, and compatibility validation. He conducted security evaluation through simulated tampering scenarios and managed bug tracking and resolution processes. Additionally, he authored the Literature Review section, positioning TrueMark within the broader research landscape.

# Chapter 6: Output – User Interface and Explanation

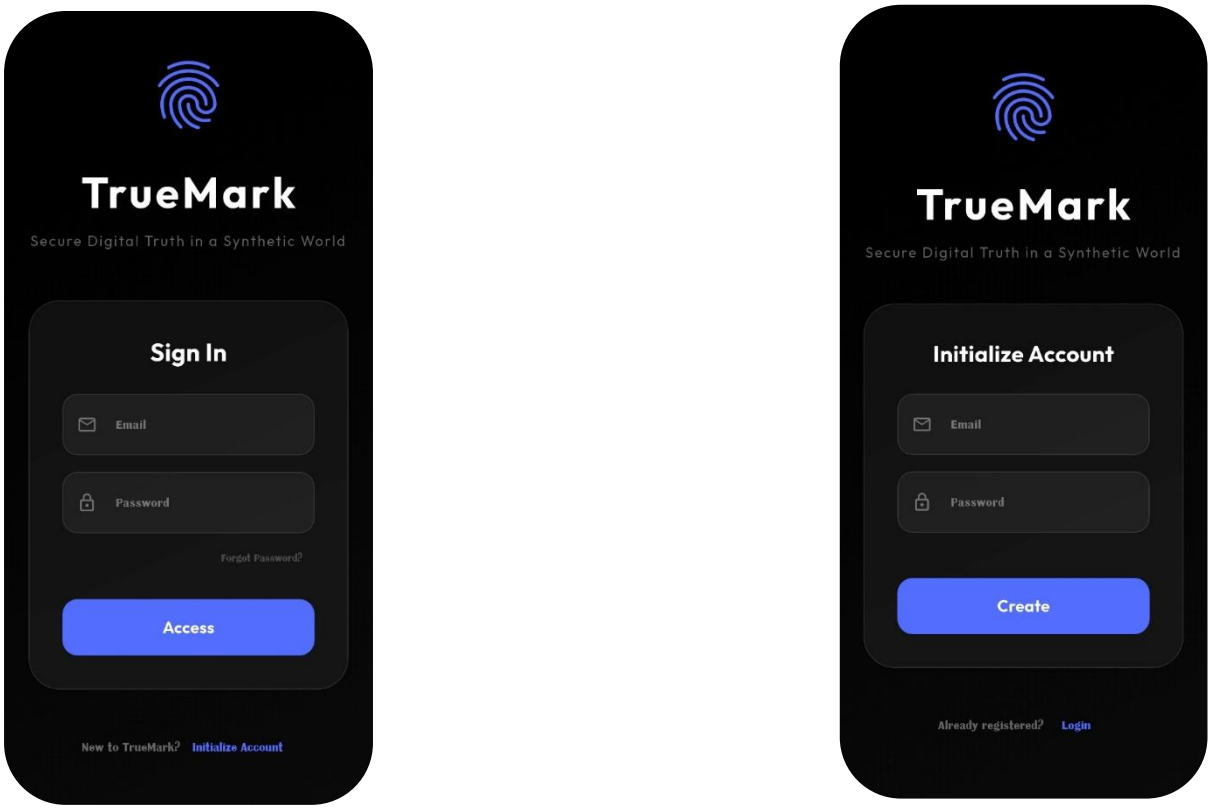
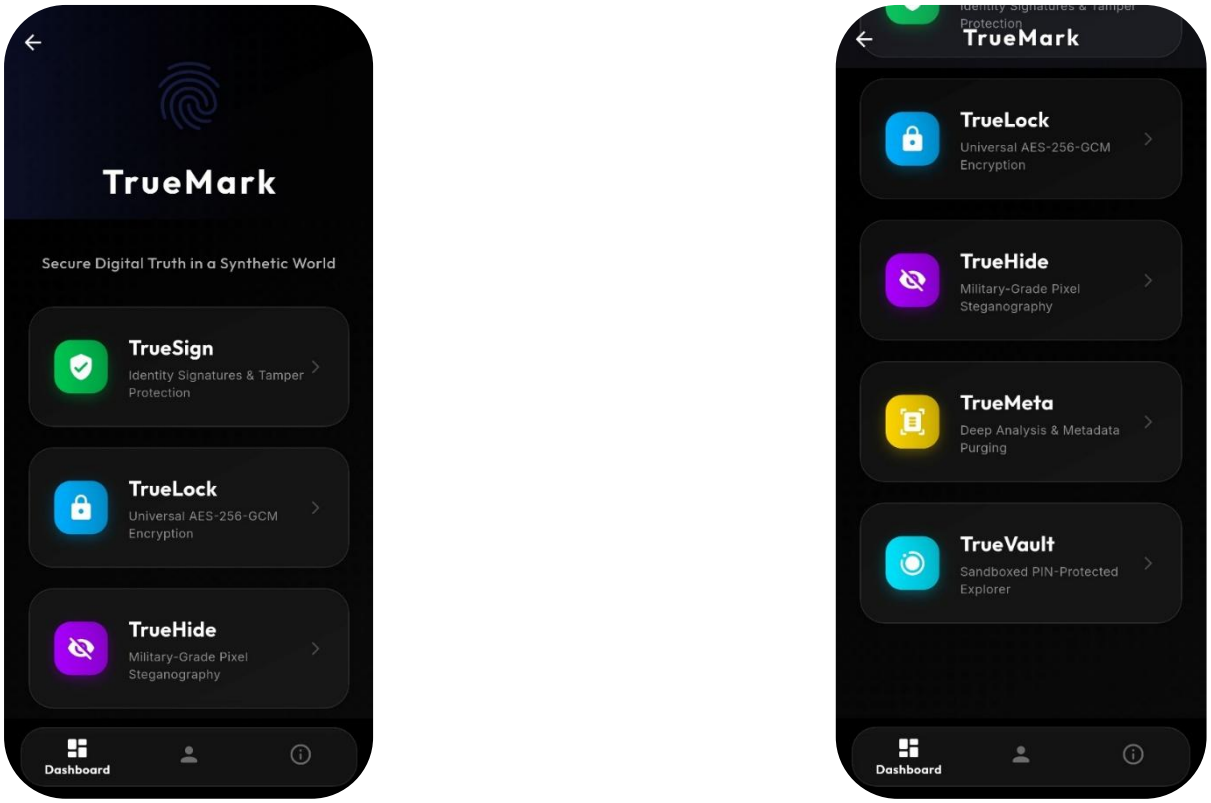



**Figure 7, 8, 9, 10 – Authentication & Dashboard**

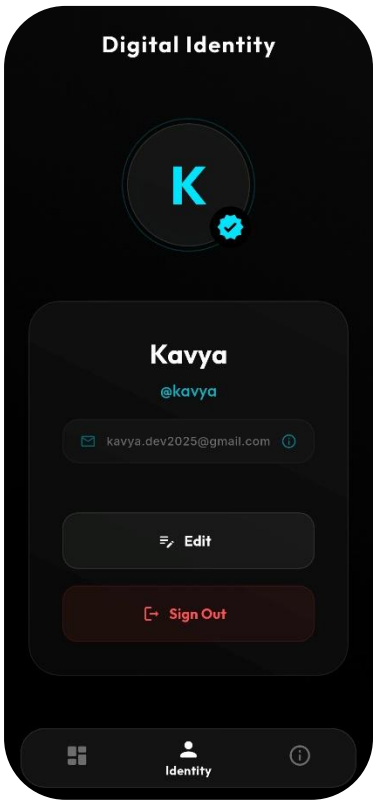


Figures 7, 8, 9, and 10 present the complete authentication workflow and the primary dashboard interface of the TrueMark application, forming the entry point into the system. These screens demonstrate how secure access control is established and how users are introduced to the application's core functionality through a structured and intuitive interface.

The authentication process begins with the Signup and Login screens, where users are required to provide their credentials using well-defined input forms. These interfaces incorporate real-time validation mechanisms to ensure correctness of data such as email format and password requirements, thereby reducing errors and improving usability. The authentication system is integrated with Firebase Authentication, enabling secure user identity management, session persistence, and seamless login handling. The design prioritizes clarity and accessibility, with immediate feedback provided for both successful and failed authentication attempts, ensuring a smooth onboarding experience.

Following successful authentication, users are directed to the dashboard, which acts as the central control hub of the TrueMark system. The dashboard is designed to provide organized access to the application's core modules, allowing users to initiate various workflows such as ownership signing, encryption, steganographic operations, metadata inspection, and secure storage. The layout follows Material Design 3 principles, utilizing a consistent Indigo-themed color scheme, clear visual hierarchy, and intuitive navigation patterns. Interactive elements are structured to guide users through the system's workflow, reducing complexity and improving efficiency.

Additionally, the dashboard supports seamless navigation between different functional components of the application, ensuring that users can transition between tasks without disruption. The overall design emphasizes both security and usability, combining strong authentication mechanisms with a user-friendly interface. These screens collectively highlight how TrueMark balances secure access control with an efficient and guided user experience, forming a reliable foundation for all subsequent operations within the system.



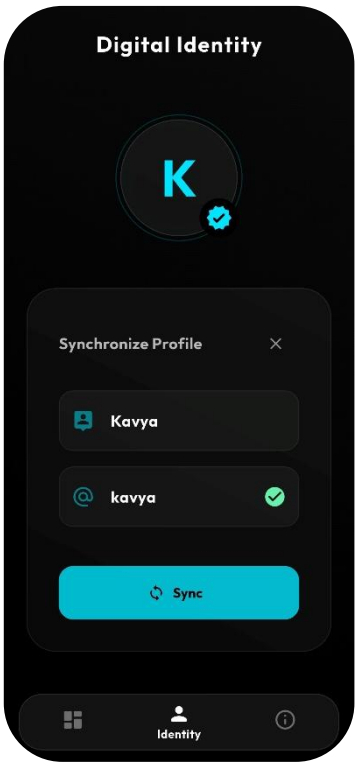
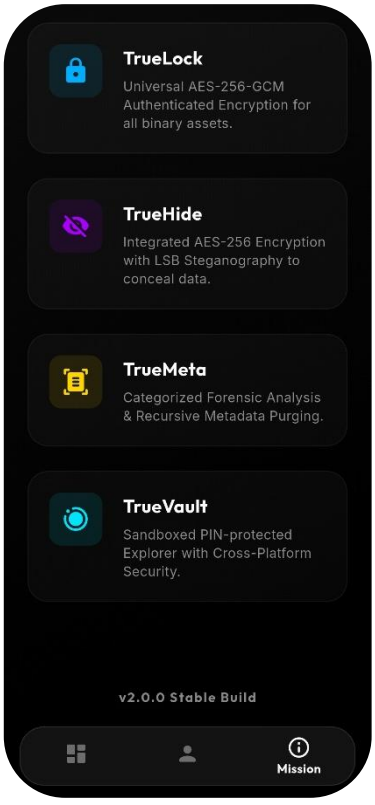




**Figure 11, 12, 13, 14 – Identity & Mission**

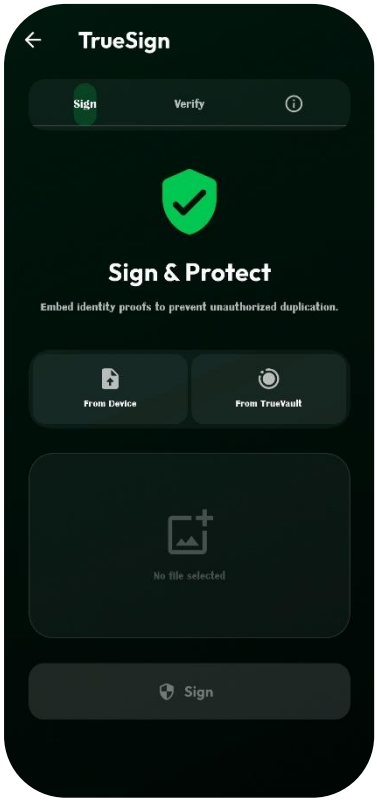
Figures 11, 12, 13, and 14 illustrate the identity setup and mission-oriented interface of the TrueMark application, highlighting how the system establishes user identity and communicates its core purpose. These screens represent a critical transition from basic authentication to personalized system interaction, ensuring that each user is uniquely identifiable within the platform's ownership and verification framework.

The identity setup component allows users to define essential profile attributes such as name and a unique username, which are subsequently linked to their account and used in ownership records within the system. This information plays a key role in the TrueSign module, where ownership metadata is embedded into media and later verified against the backend registry.

The interface ensures that identity creation is simple yet structured, with appropriate validation and guidance to maintain uniqueness and consistency of user data.

In addition to identity configuration, these screens also present the mission and conceptual foundation of TrueMark. They communicate the system's objective of establishing "Digital Truth" by addressing challenges such as content manipulation, deepfakes, and privacy risks. Through clear visual design and concise messaging, the application introduces users to its core principles—authenticity, confidentiality, and operational safety—helping them understand the purpose behind each module and workflow.

The design of these interfaces follows a clean and informative approach, combining user input elements with explanatory content to enhance user awareness and engagement. By integrating identity creation with a clear articulation of system goals, these screens ensure that users not only set up their profiles but also gain a contextual understanding of how their identity is used within the broader security ecosystem of TrueMark.



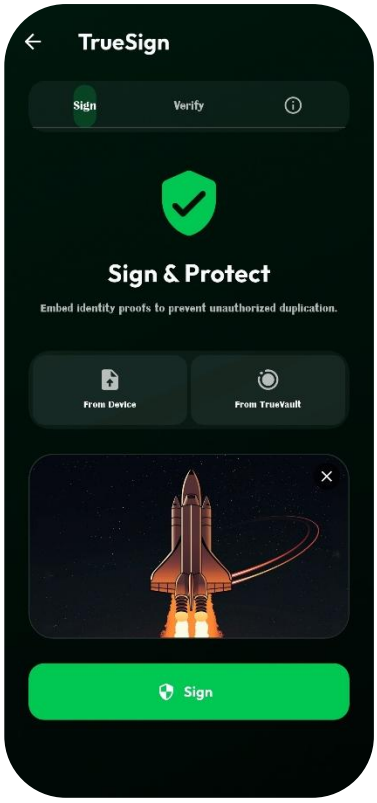
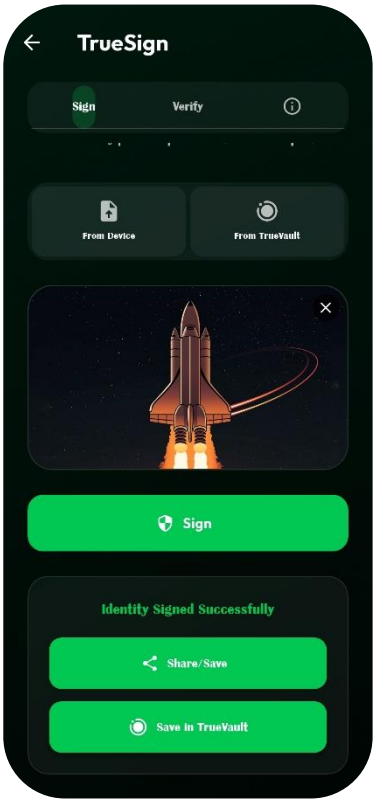




**Figure 15, 16, 17 – TrueSign Sign**



Figures 15, 16, and 17 illustrate the TrueSign signing workflow, demonstrating how ownership is embedded into digital content within the TrueMark system. These screens represent the process through which a user selects a file or image, initiates the signing operation, and generates a verifiable ownership signature that is both embedded into the media and registered in the backend.

The workflow begins with the file selection interface, where users can choose an image or supported file from their device. Once selected, the system prepares the content for processing by computing a SHA-256 hash of the file, which serves as a unique fingerprint for integrity verification. This hash, along with the user's identity and a timestamp, forms the core signature payload. The payload is then encrypted and embedded into the media using LSB steganography in the case of images, ensuring that the ownership information remains invisible and does not affect the visual quality of the file.

During the signing process, the interface provides real-time feedback to the user, indicating the progress of operations such as hashing, encryption, embedding, and registration. Upon successful completion, the system stores the ownership record in the Firestore registry using the content hash as a unique identifier. This enables fast and deterministic lookup during future verification.

The final screen in this sequence displays the output of the signing operation, confirming that the file has been successfully signed and is now associated with the user's identity. These screens collectively demonstrate how TrueMark transforms a standard media file into a verifiable digital asset, ensuring authenticity, ownership, and traceability through a seamless and user-friendly workflow.

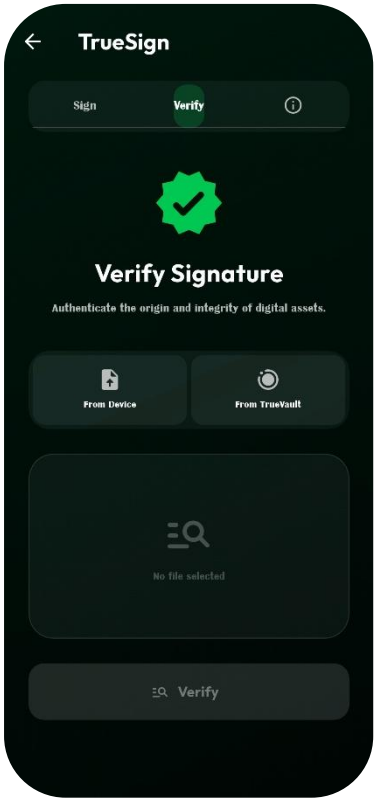
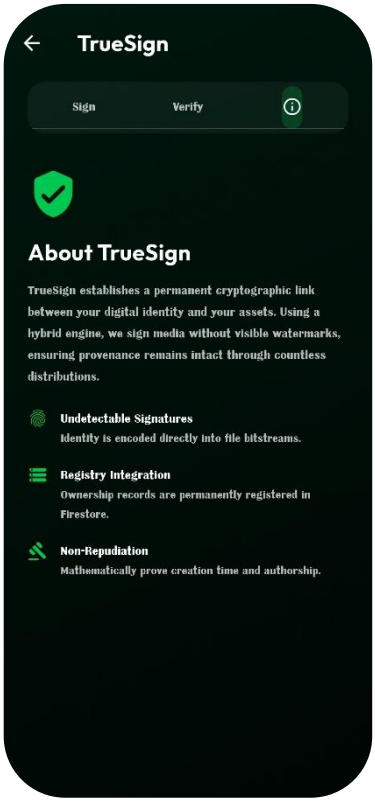




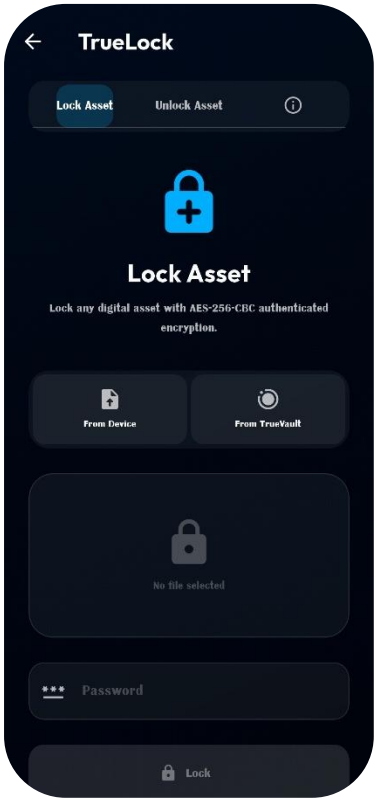
**Figure 18, 19, 20 – TrueSign Verify & About**



Figures 18, 19, and 20 illustrate the verification workflow of the TrueSign module along with the feature-specific “About” interface within the TrueMark application. These screens demonstrate how signed media is authenticated and how users are provided with contextual information specific to the TrueSign functionality.

The verification process begins with the user selecting a previously signed image or file for validation. The system extracts the embedded signature from the media using steganographic decoding and decrypts the payload to retrieve key information such as the user identifier, timestamp, and SHA-256 hash. It then recomputes the hash of the selected file and compares it with the extracted value to ensure that the content has not been altered. Following this, the application queries the Cloud Firestore registry using the hash as a unique identifier to validate ownership details. The verification result is presented through a structured interface, clearly indicating whether the file is authentic, along with relevant ownership metadata, thereby providing transparency and trust in the validation process.

The feature-specific “About” screen complements this workflow by explaining the purpose, working principles, and significance of the TrueSign module. It provides users with a concise understanding of how ownership signatures are generated, embedded, and verified within the system. This contextual guidance helps users better interpret verification results and understand the reliability of the process. Together, these screens emphasize both the functional and explanatory aspects of TrueSign, ensuring that users can not only verify digital content but also comprehend the underlying mechanism behind the verification process.



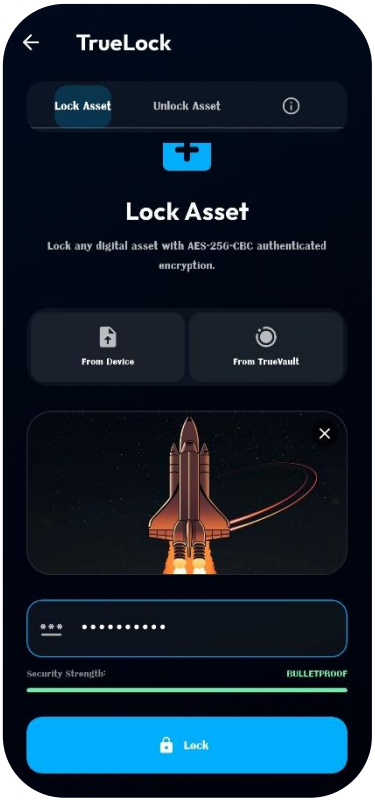
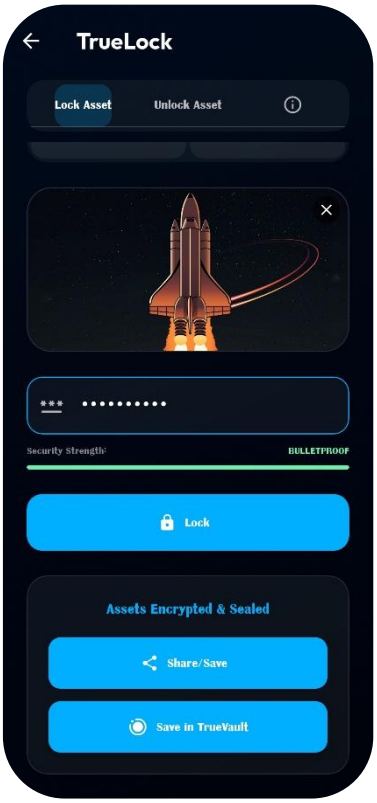




**Figure 21, 22, 23 – TrueLock Lock Asset**

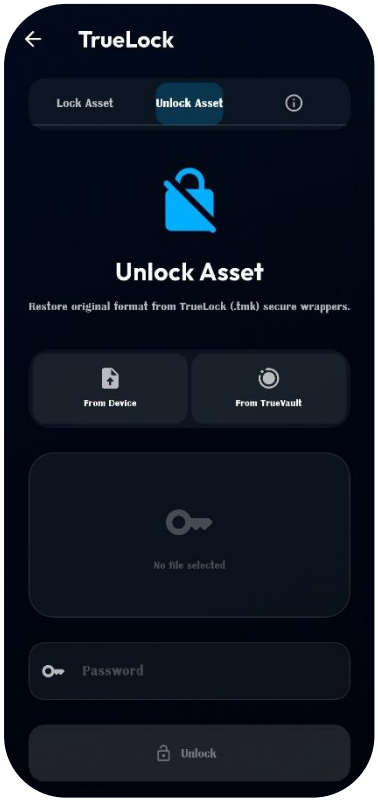


Figures 21, 22, and 23 illustrate the TrueLock “Lock Asset” workflow, demonstrating how files are securely encrypted and converted into protected containers within the TrueMark system. These screens represent the process through which a user selects a file, applies encryption, and generates a secure .tmk container for safe storage or transfer.

The workflow begins with the file selection interface, where the user chooses any supported file from the device. Once the file is selected, the system prompts the user to provide a password, which is used to derive the encryption key. The application then processes the file using AES-256-CBC encryption with PKCS7 padding, ensuring that the data is securely transformed into ciphertext. A random initialization vector (IV) is generated for each encryption operation, enhancing security by preventing predictable patterns in encrypted output.

During the locking process, the system constructs a structured container format that includes a header signature, the IV, metadata such as the original file extension, and the encrypted payload. The interface provides real-time feedback to the user, indicating the progress of encryption and ensuring transparency during processing. Upon completion, the encrypted file is saved as a .tmk container, preserving both security and recoverability.

The final screen confirms successful encryption and displays the output details, allowing the user to store or share the protected file. These screens collectively demonstrate how TrueLock enables secure file protection through a controlled and user-friendly workflow, ensuring confidentiality and integrity of sensitive data within the TrueMark ecosystem.



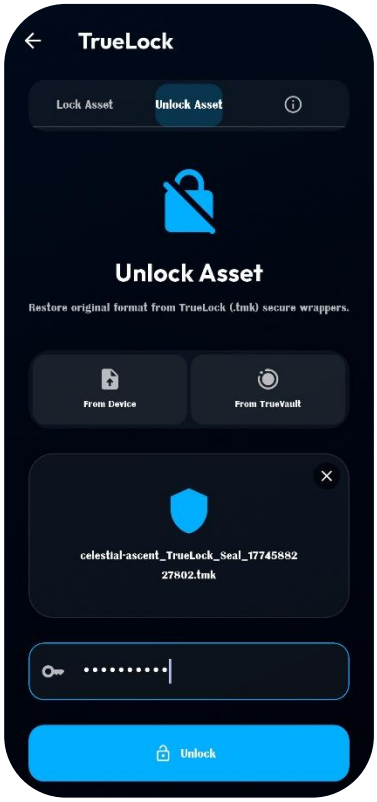
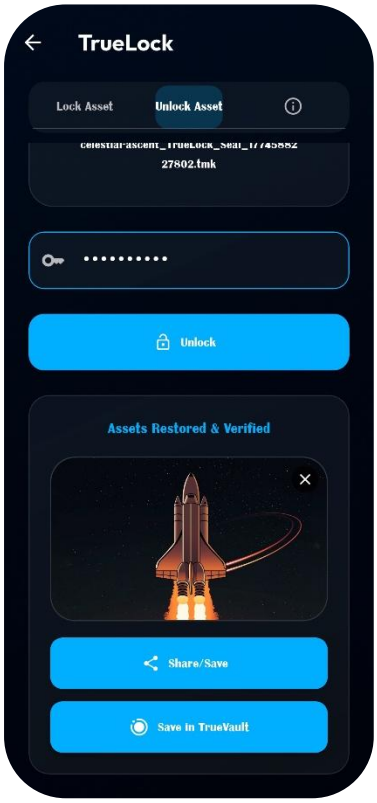
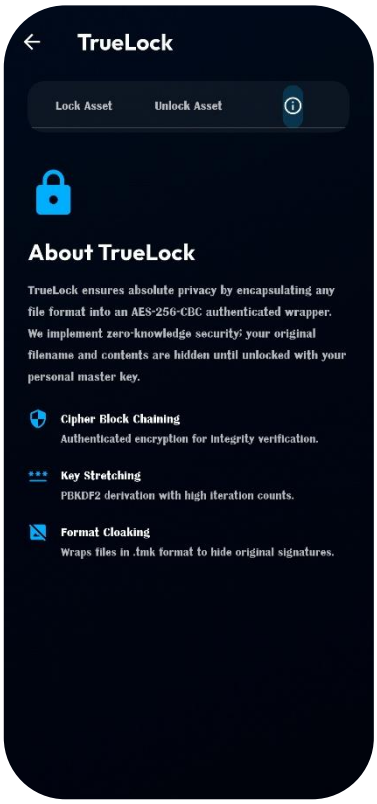




**Figure 24, 25, 26, 27 – TrueLock Unlock Asset & About**



Figures 24, 25, 26, and 27 illustrate the TrueLock “Unlock Asset” workflow along with the feature-specific “About” interface, demonstrating how encrypted .tmk containers are securely decrypted and how the system explains this functionality to the user.

The unlocking process begins with the user selecting a previously encrypted .tmk file from the device. The interface then prompts the user to enter the password that was originally used during encryption. This password is used to derive the decryption key, after which the system reads the structured container format, extracting components such as the header signature, initialization vector (IV), original file extension, and ciphertext payload. The application then performs AES-256-CBC decryption with PKCS7 padding to reconstruct the original file.

Throughout this process, the interface provides clear feedback, indicating progress and handling errors such as incorrect passwords or invalid file formats. Upon successful decryption, the original file is restored with its correct extension, ensuring usability and integrity.

The feature-specific “About” screen provides contextual information about the TrueLock module, explaining its role in secure file encryption and decryption within the TrueMark system. It outlines how files are transformed into protected containers and later restored, emphasizing the importance of password-based security and proper key management. This helps users understand both the functionality and the significance of the encryption process.

Together, these screens highlight the complete lifecycle of secure file handling in TrueLock, from encryption to decryption, while also reinforcing user understanding through integrated explanatory content.

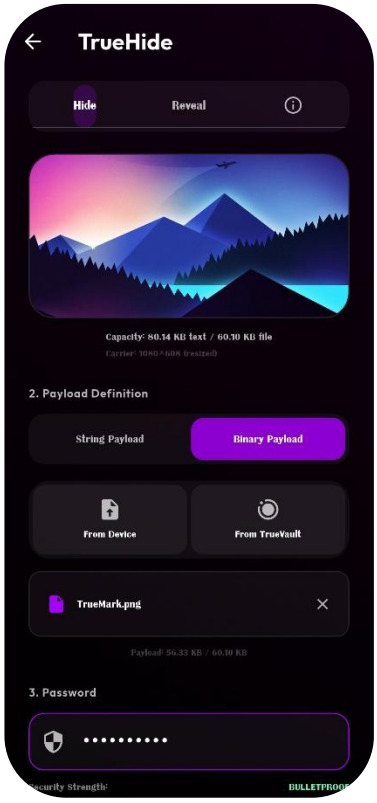
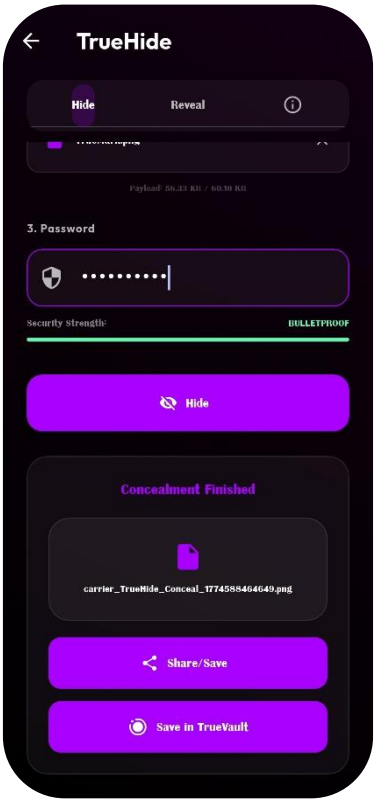




**Figure 28, 29, 30 – TrueHide Hide**

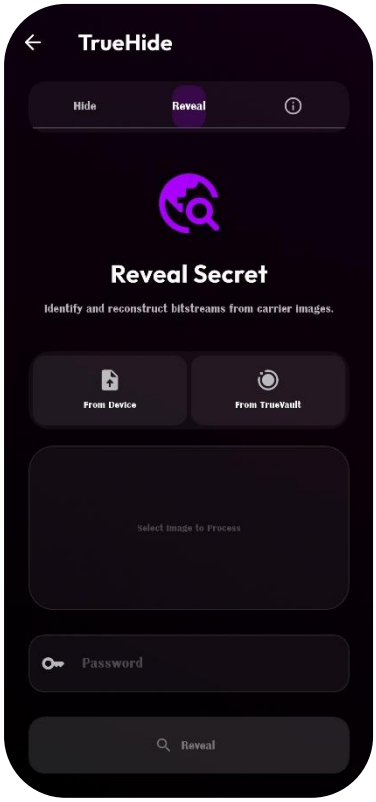


Figures 28, 29, and 30 illustrate the TrueHide “Hide” workflow, demonstrating how secret data is covertly embedded into images within the TrueMark system. These screens represent the process through which a user selects a carrier image, provides the data to be concealed, and generates an output image containing the hidden payload.

The workflow begins with the selection of a suitable carrier image from the device. The system evaluates the image and performs capacity estimation to ensure that it can accommodate the intended payload without risking corruption. Users can then input either text or select a file as the data to be hidden. Before embedding, the payload is processed and encrypted to ensure that even if detected, the concealed information remains protected. To maintain performance and stability, especially on resource-constrained devices, the application may automatically resize large images (e.g., to 1080p) to enforce predictable embedding capacity and prevent excessive memory usage.

The embedding process utilizes LSB (Least Significant Bit) steganography, where the encrypted payload is inserted into the pixel data of the image in a manner that produces no visible change to its appearance. Throughout this operation, the interface provides real-time feedback, guiding the user through each step and indicating processing status. Upon completion, a new image is generated that visually appears unchanged but contains the hidden data.

The final screen confirms successful embedding and presents the output image, which can then be stored or shared. These screens collectively demonstrate how TrueHide enables low-visibility data concealment through a controlled and user-friendly workflow, combining steganographic techniques with encryption for enhanced security.



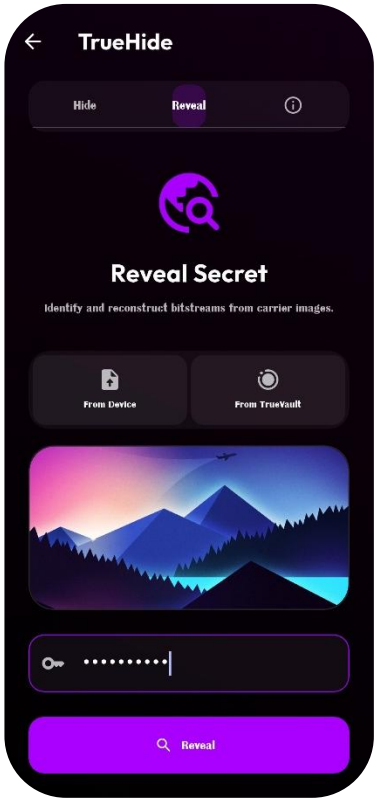
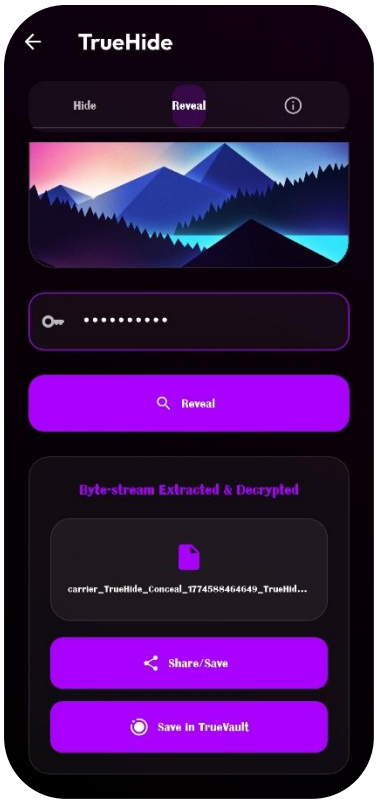
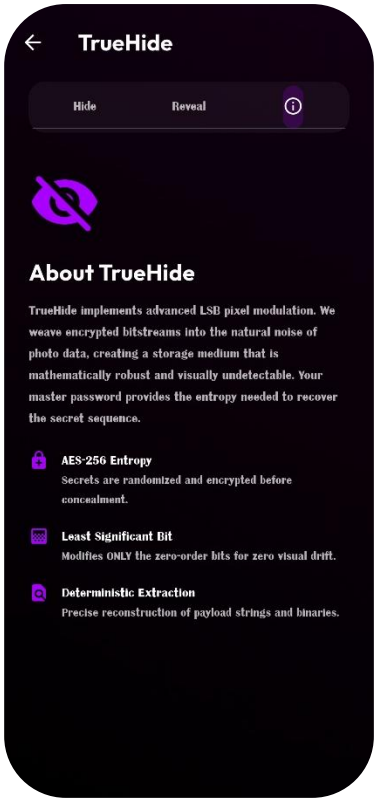




**Figure 31, 32, 33, 34 – TrueHide Reveal & About**



Figures 31, 32, 33, and 34 illustrate the TrueHide “Reveal” workflow along with the feature-specific “About” interface, demonstrating how hidden data is securely extracted from carrier images and how the system explains this functionality to the user.

The reveal process begins with the user selecting an image that potentially contains concealed data. The system analyzes the image and applies steganographic decoding techniques to extract the embedded bitstream from the pixel data. Using the appropriate key, the extracted payload is then decrypted and reconstructed into its original form, which may be either text or a binary file. The interface guides the user through this process, providing clear feedback on extraction status and handling cases such as invalid inputs or absence of hidden data. Upon successful extraction, the recovered content is displayed or made available for download, ensuring that the original information is accurately restored.

In addition to the functional workflow, the feature-specific “About” screen provides a concise explanation of the TrueHide module, outlining how data is embedded and later retrieved using steganographic and cryptographic techniques. It helps users understand the purpose, limitations, and correct usage of the feature, reinforcing awareness of secure data handling practices.

Together, these screens highlight the complete lifecycle of covert data communication within TrueHide, from hidden payload recovery to user guidance, ensuring both functional reliability and user comprehension of the underlying process.



**Figure 35, 36, 37, 38 – TrueMeta Analyze**


Figures 35, 36, 37, and 38 illustrate the TrueMeta “Analyze” workflow, demonstrating how the TrueMark system performs metadata inspection on media files to reveal hidden information. These screens represent the process through which a user selects an image or file and the system extracts and presents embedded metadata in a structured and readable format.

The workflow begins with file selection, where the user chooses a media file for analysis. Once selected, the system processes the file and extracts metadata such as EXIF information, including device details, timestamps, image properties, and, where available, location data. The extracted information is then categorized and displayed through an organized interface, allowing users to easily interpret the data. This structured presentation helps users understand what hidden information is present within their media, which is often not visible through standard viewing methods.

The interface emphasizes clarity and readability by grouping metadata into logical sections, ensuring that technical details are accessible even to non-expert users. Real-time feedback is provided during the analysis process, and the results are displayed without altering the original file, maintaining its integrity.

These screens highlight the forensic capabilities of the TrueMeta module, enabling users to inspect and understand the hidden data associated with their media. This step plays a crucial role in raising awareness about potential privacy risks and serves as a precursor to metadata sanitization operations within the TrueMark system.

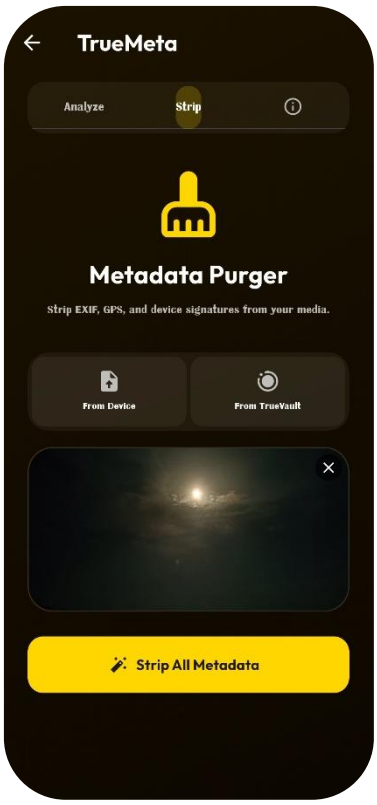
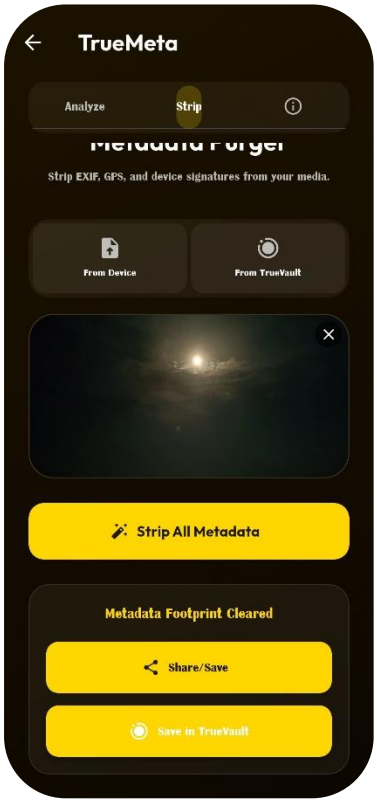
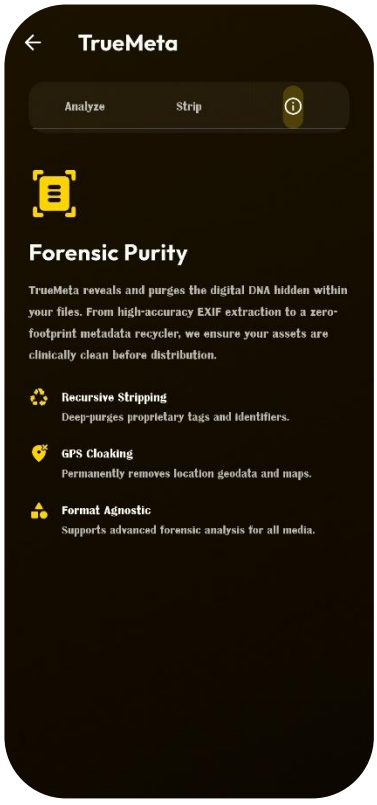




**Figure 39, 40, 41, 42 – TrueMeta Strip & About**



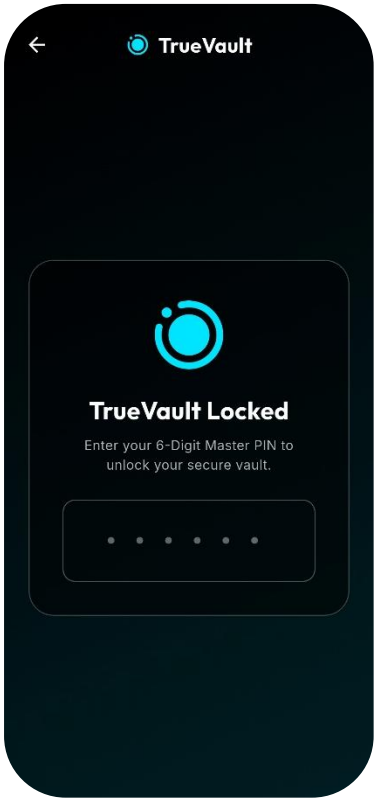



Figures 39, 40, 41, and 42 illustrate the TrueMeta “Strip” workflow along with the feature-specific “About” interface, demonstrating how hidden metadata is removed from media files and how the system explains this functionality to the user.

The stripping process begins with the user selecting a media file that has already been analyzed or is intended for sanitization. The system processes the file by decoding its pixel data and re-encoding it to eliminate embedded metadata such as EXIF information, GPS coordinates, device identifiers, and textual tags. This approach ensures that all hidden metadata is effectively removed while preserving the visual quality and usability of the file. The interface provides clear feedback during the operation, indicating progress and confirming successful completion of the sanitization process. Upon completion, a new output file is generated with a clean metadata profile and a distinct naming convention to differentiate it from the original.

The feature-specific “About” screen provides contextual information about the TrueMeta module, explaining the importance of metadata sanitization in protecting user privacy. It outlines how hidden information can unintentionally expose sensitive details and how the stripping process mitigates such risks. This helps users understand both the functionality and the significance of the feature within the overall system.

Together, these screens highlight the practical application of metadata sanitization in TrueMark, combining effective data cleaning with user awareness, and reinforcing the system’s role in enhancing privacy and reducing unintended information leakage.



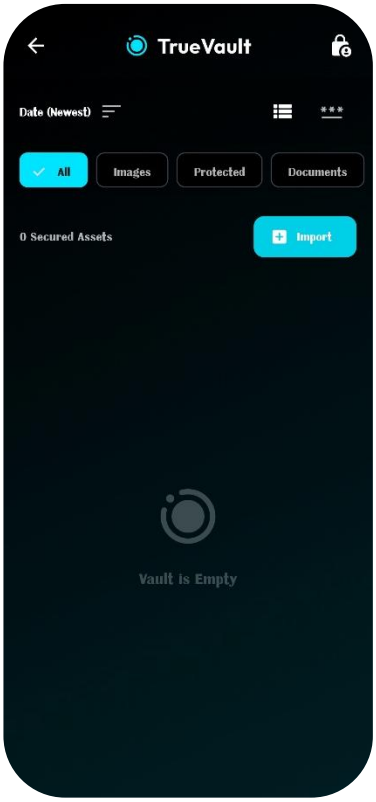
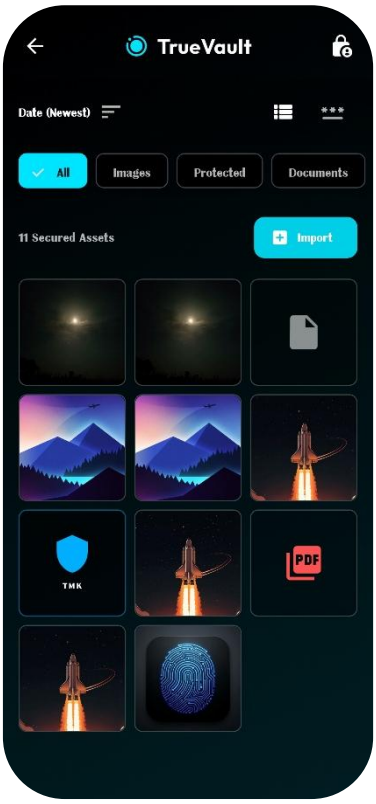




**Figure 43, 44, 45 – TrueVault Empty & Media**



Figures 43, 44, and 45 illustrate the TrueVault interface in both its empty and populated states, demonstrating how the system manages secure storage of user-generated artifacts. These screens represent the transition from an initial state, where no files are stored, to an active state where encrypted, processed, or generated media is organized and accessible within the vault.

The empty state screen provides a clean and informative interface indicating that no files are currently stored in the vault. It typically includes visual cues and guidance to inform the user about the purpose of TrueVault and how files generated from various modules—such as signed images, encrypted .tmk containers, or sanitized media—will appear once stored. This design helps users understand the role of the vault as a centralized and secure repository within the application.

In contrast, the populated state screens display stored media in an organized layout, such as grid or list views, allowing users to browse and manage their files efficiently. Files are categorized based on type, such as images, documents, or protected assets, and may include sorting and filtering options to enhance usability. The system ensures that all stored files are saved within the application's private storage space, maintaining isolation from external access.

Additionally, file operations are handled atomically to prevent corruption, ensuring reliability during save and retrieval processes.

These screens collectively demonstrate how TrueVault provides a controlled and user-friendly environment for managing sensitive digital artifacts, supporting both usability and security through structured storage and intuitive interaction.

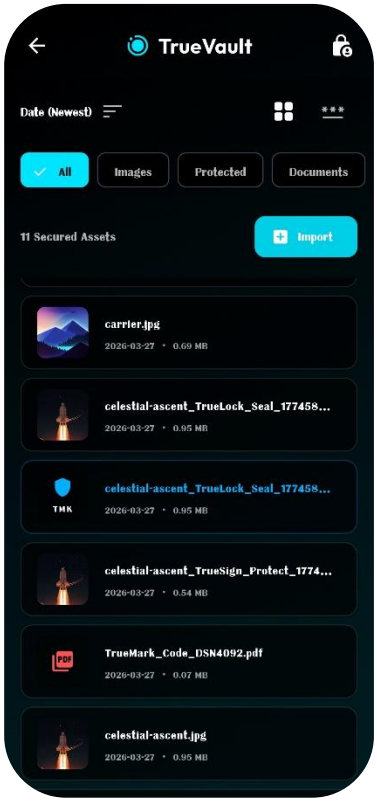
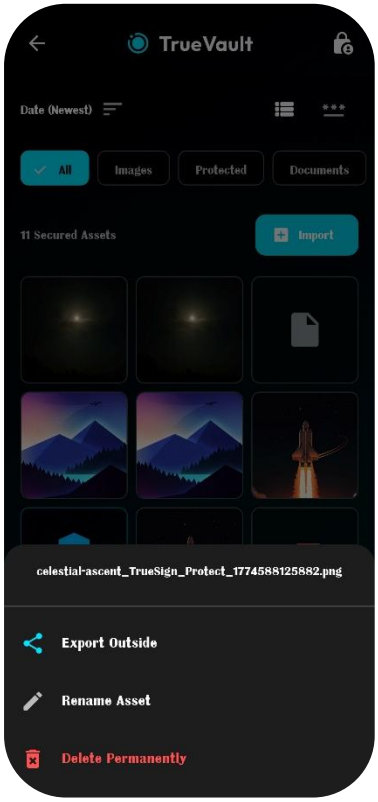
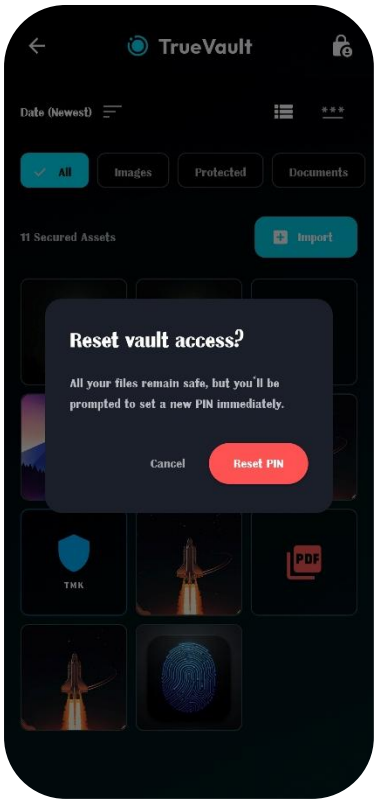




**Figure 46, 47, 48, 49 – TrueVault Filter, Sort, View, File Options & Reset PIN**



Figures 46, 47, 48, and 49 illustrate the advanced interaction features of the TrueVault module, showcasing filtering, sorting, file viewing, file-specific options, and the PIN reset functionality. These screens demonstrate how the vault evolves from a basic storage interface into a fully manageable and user-controlled environment for handling sensitive digital artifacts.

The filtering and sorting interfaces allow users to efficiently organize stored files based on categories such as images, documents, or protected assets, as well as ordering criteria like name, date, or type. This improves usability, especially as the number of stored items increases, enabling quick retrieval and better management of data. The viewing interface provides a detailed representation of selected files, allowing users to preview content and access relevant information in a structured manner without compromising security.

In addition to viewing capabilities, the file options interface offers actions such as sharing, exporting, or deleting stored items, giving users complete control over their data within the secure environment. These operations are designed to be safe and controlled, ensuring that file integrity is maintained during any interaction.

The PIN reset functionality represents an important security feature of TrueVault, allowing users to update their authentication credentials while maintaining secure access control. This process is handled through secure storage mechanisms, ensuring that authentication data remains protected at the system level.

Together, these screens highlight the flexibility and robustness of the TrueVault module, combining secure storage with advanced management capabilities and user-centric controls, thereby enhancing both usability and security within the TrueMark system.

# Chapter 7: Conclusion

This capstone project successfully designed and implemented TrueMark, a cross-platform digital forensics and privacy suite that integrates cryptography, steganography, and metadata intelligence to establish authenticity, confidentiality, and operational safety in digital media. The system demonstrates the feasibility of a client-side, workflow-driven security model where complex operations are abstracted into an intuitive user experience. Users can sign media to embed ownership information, encrypt files into secure containers, conceal data within images, analyze and remove metadata, and manage outputs within a secure vault, all within a unified application environment.

The implemented system validates the effectiveness of a layered security architecture combining multiple complementary techniques. TrueMark integrates SHA-256-based content fingerprinting, AES-256-CBC encryption with PKCS7 padding, LSB-based steganographic embedding, and a cloud-backed ownership registry using Firebase Firestore. Unlike isolated implementations commonly found in literature, the system provides an end-to-end pipeline that connects identity, protection, verification, and storage into a single coherent workflow. The use of content-based hashing as a primary identifier enables efficient and deterministic verification, while the Firestore registry introduces a practical provenance layer that extends beyond standalone cryptographic validation. The modular design ensures separation of concerns across frontend, backend, and processing layers, supporting maintainability and extensibility.

Despite its functional completeness, the current implementation presents certain limitations that restrict its robustness in real-world deployment scenarios. The use of spatial-domain LSB steganography makes embedded data vulnerable to transformations such as compression, resizing, or format conversion, which are common in real-world media sharing platforms. The encryption model, while secure in principle, relies on password-based mechanisms that require careful user practices for strong security. Additionally, the absence of asymmetric digital signatures limits independent third-party verification capabilities, which are important for high-trust environments such as legal or journalistic applications. The reliance on a centralized Firestore registry also introduces a dependency on backend availability and trust.

These limitations highlight key principles in security system design. Practical deployment requires not only strong algorithms but also robustness against real-world transformations, proper key management, and support for independent verification. The project demonstrates that while client-side security mechanisms can effectively protect and verify content, achieving production-level reliability requires enhancements such as transformation-resistant watermarking, stronger key derivation techniques, and decentralized or cryptographic identity models.

The system architecture confirms that proposed enhancements are technically feasible within the existing framework. Future improvements may include advanced watermarking techniques in the frequency domain for improved robustness, integration of digital signatures for stronger authenticity guarantees, and AI-based analysis for detecting synthetic or manipulated media. Additional enhancements such as biometric authentication, decentralized verification mechanisms, and expanded platform support can further strengthen the system’s applicability and usability.

From a broader perspective, TrueMark addresses emerging challenges in digital content authenticity, privacy protection, and secure data handling in an era increasingly influenced by synthetic media and automated content generation. By combining multiple security approaches into a single accessible platform, the system demonstrates how complex protection mechanisms can be made usable for general users. At the same time, it highlights that technical solutions must be complemented by user awareness and evolving standards to achieve widespread effectiveness.

The project provides significant educational value by integrating concepts from cryptography, steganography, mobile application development, cloud services, and user interface design into a single system. The documented architecture, implementation strategies, and identified limitations offer a strong foundation for further research and development in digital media protection systems.

In conclusion, TrueMark successfully achieves its objective of demonstrating a practical, integrated approach to digital authenticity and privacy on modern computing platforms. The working system validates that secure, multi-layered protection can be implemented efficiently while maintaining usability. Although certain limitations remain, the modular architecture and clearly defined enhancement pathways provide a solid foundation for future development. As concerns around digital trust, content ownership, and data privacy continue to grow, systems like TrueMark represent an important step toward reliable and accessible solutions in the evolving digital landscape.

# Chapter 8: References


[1] Sharma, P., & Singh, A. (2019). A Secure Method for Image Signaturing using SHA256, RSA and Advanced Encryption Standard. *International Journal of Computer Applications*, 182(15), 25-30. https://doi.org/10.5120/ijca2019918234

[2] Kumar, R., & Chand, S. (2021). A Study on Digital Signatures with Privacy Factor. *Journal of Cryptographic Engineering*, 11(3), 245-258. https://doi.org/10.1007/s13389-021-00265-3

[3] Qureshi, M. A., & Deriche, M. (2020). A Comprehensive Survey on Methods for Image Integrity and Authentication. *IEEE Access*, 8, 43416-43442. https://doi.org/10.1109/ACCESS.2020.2977396

[4] Al-Haj, A., & Abandah, G. (2022). A Cryptographic Framework for Secure Medical Imaging in Smart Healthcare Environments. *Journal of Medical Systems*, 46(8), 1-15. https://doi.org/10.1007/s10916-022-01845-2

[5] Mahmood, Z., & Khan, M. A. (2020). Securing Medical Images for Mobile Health Systems Using a Combined Approach of Encryption and Steganography. In *Proceedings of the International Conference on Advanced Computing and Communication Systems*, 234-239. IEEE. https://doi.org/10.1109/ICACCS48705.2020.9074184

[6] Zhang, Y., Wang, J., & Chen, X. (2021). Privacy-Preserving Biometrics Image Encryption and Digital Signature Technique Using Arnold and ElGamal. *Multimedia Tools and Applications*, 80(12), 18693-18717. https://doi.org/10.1007/s11042-021-10643-7

[7] Rodriguez, M., & Thompson, L. (2024). The Evolution of Digital Signature Technologies in Mobile Devices. *ACM Computing Surveys*, 56(4), 1-38. https://doi.org/10.1145/3631527

[8] Hassan, F. S., & Gutub, A. (2022). Secure Mobile Computing Authentication Utilizing Hash, Cryptography and Steganography Combination. *Journal of King Saud University - Computer and Information Sciences*, 34(6), 2811-2823. https://doi.org/10.1016/j.jksuci.2022.03.009

[9] Google LLC. (2023). *Flutter Documentation: Building Beautiful Native Apps*. Retrieved from https://docs.flutter.dev/

[10] Dart Team. (2023). *Dart Programming Language Specification* (Version 3.0). Retrieved from https://dart.dev/guides/language/spec

[11] Firebase. (2023). *Firebase Documentation: Authentication, Firestore, and Storage*. Google Cloud. Retrieved from https://firebase.google.com/docs

[12] National Institute of Standards and Technology (NIST). (2001). *Advanced Encryption Standard (AES)* (FIPS PUB 197). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.197

[13] National Institute of Standards and Technology (NIST). (2015). *Secure Hash Standard (SHS)* (FIPS PUB 180-4). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.180-4

[14] Rivest, R. L., Shamir, A., & Adleman, L. (1978). A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. *Communications of the ACM*, 21(2), 120-126. https://doi.org/10.1145/359340.359342

[15] Koblitz, N. (1987). Elliptic Curve Cryptosystems. *Mathematics of Computation*, 48(177), 203-209. https://doi.org/10.1090/S0025-5718-1987-0866109-5

[16] Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., & Yang, B. Y. (2012). High-Speed High-Security Signatures. *Journal of Cryptographic Engineering*, 2(2), 77-89. https://doi.org/10.1007/s13389-012-0027-1

[17] Dworkin, M. J. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC* (NIST SP 800-38D). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38D

[18] Cox, I. J., Miller, M. L., Bloom, J. A., Fridrich, J., & Kalker, T. (2008). *Digital Watermarking and Steganography* (2nd ed.). Morgan Kaufmann Publishers. ISBN: 978-0123725851

[19] Petitcolas, F. A. P., Anderson, R. J., & Kuhn, M. G. (1999). Information Hiding—A Survey. *Proceedings of the IEEE*, 87(7), 1062-1078. https://doi.org/10.1109/5.771065

[20] Adobe Systems. (2023). *Content Authenticity Initiative: Technical Specifications for C2PA Standard* (Version 1.3). Coalition for Content Provenance and Authenticity. Retrieved from https://c2pa.org/specifications/specifications/1.3/

[21] Kaliski, B. (2000). *PKCS #5: Password-Based Cryptography Specification Version 2.0* (RFC 2898). Internet Engineering Task Force. https://doi.org/10.17487/RFC2898

[22] McGrew, D. A., & Viega, J. (2004). The Galois/Counter Mode of Operation (GCM). *Submission to NIST Modes of Operation Process*, 1-43. Retrieved from https://csrc.nist.gov/CSRC/media/Projects/Block-Cipher-Techniques/documents/BCM/proposed-modes/gcm/gcm-spec.pdf

[23] Fridrich, J. (2009). *Steganography in Digital Media: Principles, Algorithms, and Applications*. Cambridge University Press. ISBN: 978-0521190190

[24] Kessler, G. C. (2007). An Overview of Steganography for the Computer Forensics Examiner. *Forensic Science Communications*, 6(3). Retrieved from https://www.garykessler.net/library/steganography.html

[25] Jehl, G., & others. (2016). *ExifTool Application Documentation*. Phil Harvey. Retrieved from https://exiftool.org/

**https://github.com/JitSurani/TrueMark**